

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Институт естественных и точных наук
Кафедра системного программирования

**Разработка приложения для решения задач
по идентификации объектов на изображениях**

КУРСОВАЯ РАБОТА

по дисциплине «Программная инженерия»
ЮУрГУ – 02.03.02.2026.308-031.КР

Нормоконтролер,
Доцент кафедры СП, к.ф.-м.н.
_____ Я.А. Краева
«__»_____ 2026 г.

Научный руководитель:
Доцент кафедры СП, к.ф.-м.н.
_____ Я.А. Краева

Автор работы:
студент группы ЕТ-302
_____ А.М. Доблер

Работа защищена
с оценкой: _____
«__»_____ 2026 г.

Челябинск, 2026 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Институт естественных и точных наук
Кафедра системного программирования

УТВЕРЖДАЮ
Зав. кафедрой СП
_____ Л.Б. Соколинский
«__»_____.2026 г.

ЗАДАНИЕ
на выполнение курсовой работы
по дисциплине «Программная инженерия»
студенту группы ЕТ-302 Доблер Александру Максимовичу,
обучающемуся по направлению 02.03.02 «Фундаментальная информатика и инфор-
мационные технологии»

1. Тема работы

Разработка приложения для решения задач по идентификации объектов на изображениях.

2. Срок сдачи студентом законченной работы: 05.06.2026 г.

3. Исходные данные к работе

3.1. Fernandez A.V. Mastering OpenCV 4 with Python. 2024. 532 с.

3.2. Галлатин К., Элбон К. Машинное обучение с использованием Python.
Сборник рецептов. 2023. 416 с.

3.3. Журнал Молодой ученый – Методы определения объектов на изображе-
нии / Я.Э.Живрин, Н.Б.Алкзир [Электронный ресурс] URL:
<https://moluch.ru/archive/193/48447/> (дата обращения: 19.02.2026 г.).

4. Перечень подлежащих разработке вопросов

- 4.1. Провести анализ предметной области и обзор существующих решений.
- 4.2. Сформулировать требования к десктопному приложению.
- 4.3. Спроектировать десктопное приложение.
- 4.4. Реализовать десктопное приложение.
- 4.5. Провести тестирование разработанного десктопное приложения.

5. Дата выдачи задания: 2 февраля 2026 г.

Научный руководитель:

Доцент кафедры СП, к.ф.-м.н.

Краева Я.А.

Задание принял к исполнению:

Доблер А.М.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Обзор литературы	9
1.2. Анализ работы YOLO.....	13
1.3. Анализ работы OCR.....	17
1.4. Анализ аналогичных проектов	22
2. ПРОЕКТИРОВАНИЕ	27
2.1. Требования к системе	27
2.2. Варианты использования	31
2.3. Архитектура приложения.....	33
2.4. Диаграммы деятельности и последовательности	35
3. РЕАЛИЗАЦИЯ	38
3.1. Обоснование выбора инструментов разработки	38
3.2. Архитектурная структура и распределение модулей проекта	39
4. ТЕСТИРОВАНИЕ	47
4.1. Автоматизированное тестирование	47
4.2. Функциональное тестирование	48
ЗАКЛЮЧЕНИЕ	49
ЛИТЕРАТУРА	50
ПРИЛОЖЕНИЯ.....	53
Приложение А. Спецификация вариантов использования.....	53
Приложение Б. Результаты функционального тестирования.....	63

ВВЕДЕНИЕ

Актуальность

В условиях экспоненциального роста объемов визуальных данных автоматическая идентификация объектов на изображениях стала критически важной технологией для беспилотного транспорта, систем безопасности, медицинской диагностики и производственного контроля.

Основу современных решений в этой области составляют методы компьютерного зрения и глубокого обучения, в частности, сверточные нейронные сети (CNN). Несмотря на обилие готовых архитектур (YOLO, Faster R-CNN, Mask R-CNN), их адаптация к узкоспециализированным прикладным задачам требует дообучения для устранения артефактов распознавания. Кроме того, внедрение таких алгоритмов в реальную практику невозможно без создания интуитивно понятного графического интерфейса. Таким образом, разработка законченного десктопного приложения, интегрирующего SOTA-алгоритмы в удобный функциональный инструмент, является актуальной инженерной задачей.

Постановка задачи

Целью курсовой работы является разработка десктопного приложения для решения задач по идентификации и детекции объектов на изображениях. Для достижения поставленной цели необходимо решить следующие задачи:

- 1) выполнить анализ предметной области и обзор существующих аналогов;
- 2) сформулировать требования к десктопному приложению;
- 3) спроектировать десктопное приложение;
- 4) реализовать десктопное приложение;
- 5) провести тестирование разработанного десктопного приложения.

Структура и содержание работы

Работа состоит из введения, четырех глав, заключения, списка литературы и приложений. Объем работы составляет 63 страниц, объем списка литературы – 19 источников.

В первой главе «Анализ предметной области» рассмотрены существующие программные решения для компьютерного зрения и оптического распознавания символов, выявлены их функциональные возможности, достоинства и недостатки. На основе обзора обоснован выбор технологического стека, библиотек глубокого обучения и архитектурного решения.

Во второй главе «Проектирование» сформулированы функциональные и нефункциональные требования к системе, представлены диаграмма вариантов использования, диаграмма компонентов, диаграммы деятельности и последовательности, а также описана структура локальной базы данных.

В третьей главе «Реализация» описана разработка графического интерфейса пользователя на базе объектно-ориентированного фреймворка, модулей асинхронного инференса нейросетевых моделей и подсистемы генерации отчетности.

В четвертой главе «Тестирование» представлены результаты функционального и автоматизированного тестирования, разработанного десктопного приложения.

В заключении приведены итоги работы и сформулированы возможные направления дальнейшего развития системы.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Предметной областью работы является процесс автоматизированного анализа графических данных с целью локализации и распознавания, содержащихся на них объектов и текстовых элементов. В рамках этого процесса оператор предоставляет системе исходные медиафайлы, а программный комплекс выполняет их предварительную обработку, сквозной инференс нейросетевых моделей и структурирование извлеченной информации. Разрабатываемая система автоматизирует данные операции, предоставляя пользователю интерактивные инструменты для верификации результатов, генерации локальных датасетов и формирования экспертной отчетности. В современной инженерной практике автоматизация подобных процессов базируется на ряде фундаментальных подходов. Ниже представлена сводка основных парадигм, их принципов работы и решаемых задач.

Многослойные перцептроны (MLP) – полносвязные сети, оперирующие с векторными представлениями изображений, полученными путем растягивания тензора пикселей. Отсутствие пространственной индуктивного смещения (inductive bias) приводит к экспоненциальному росту параметров, неинвариантности к пространственным преобразованиям и практической неприменимости для задач с высоким разрешением.

Сверточные нейронные сети (CNN) вводят три ключевых индуктивных смещения: локальность связей (фильтры ядра $K \times K$), трансляционную эквивариантность (разделяемые веса) и иерархическую композиционность. Типичный блок включает операцию дискретной свертки (Conv2D), поэлементную нелинейную активацию (ReLU) и операцию субдискретизации (пулинг). Архитектурная эволюция (VGG, Inception, ResNet) решала проблемы оптимизации глубоких сетей через остаточные связи (skip-connections) и эффективные блочные конструкции.

Детекторы объектов реализуют две методологии. Двухэтапные (R-CNN, Faster R-CNN) используют каскад: Region Proposal Network генерирует области интереса (RoI), после чего головы классификации и регрессии

уточняют категории и bounding box. Одноэтапные детекторы (YOLO, SSD) применяют плотное предсказание на анкер-боксах в однопроходной архитектуре, достигая баланса между precision и inference time.

Сети семантической сегментации (FCN, U-Net) построены по энкодер-декодерной схеме с сохранением пространственной согласованности. Энкодер осуществляет сжатие пространственного разрешения с увеличением глубины признаков, декодер выполняет транспонированную свертку или билинейную интерполяцию для апсемплинга. Skip-connections обеспечивают сшивание низкоуровневых (границы) и высокоуровневых (семантика) признаков, минимизируя потерю информации при пулинге.

Генеративные модели оперируют в латентных пространствах. GAN минимизируют дивергенцию Дженсена-Шеннона между распределениями реальных и сгенерированных данных через адверсариальную игру генератора G и дискриминатора D . VAE максимизируют evidence lower bound (ELBO), обучая вероятностный энкодер $Q(z|X)$ и декодер $P(X|z)$ с применением reparameterization trick для дифференцируемого сэмплирования из априорного распределения.

Архитектуры внимания и трансформеры (ViT) заменяют локальные свертки глобальным механизмом self-attention. Изображение разбивается на N непересекающихся патчей, которые линейно проецируются в эмбединги с добавлением позиционных кодировок. Много голов внимания вычисляют взвешенные суммы значений на основе парной аффинности между запросами и ключами, моделируя долгосрочные зависимости. Иерархические варианты (Swin Transformer) вводят локальные окна и сдвигаемые оконные внимания для снижения вычислительной сложности $O(N^2)$ до $O(N)$.

Современные state-of-the-art гибридные архитектуры (ConvNeXt, Hybrid ViT) комбинируют сильные стороны CNN (индуктивное смещение локальности, эффективность) и трансформеров (моделирование глобального контекста, масштабируемость), демонстрируя эффективность на задачах классификации, детекции и сегментации в условиях больших данных

1.1. Обзор литературы

Орлов. А. А. Применение преобразования Хоха для обнаружения и идентификации объектов на изображениях [15]

В работе Орлова представлен алгоритм Хоха, который направлен на решение задачи идентификации объектов с минимальными затратами вычислительных ресурсов. В основе лежит контурный анализ с использованием модифицированного преобразования Хоха (Хафа). Это подход эпохи, предшествующей глубокому обучению.

Статья предлагает метод обнаружения объектов по их контуру. Изображение преобразуется в градиент, важные границы выделяются автоматическим порогом. Контурные точки переводятся в инвариантное к масштабу и вращению пространство параметров (угловая кривизна и разность углов). Обнаружение и идентификация выполняются путем сравнения «сигнатур» – аккумуляторных массивов в этом пространстве.

Метод обеспечивает формальную инвариантность к сдвигу, масштабу и повороту объекта. Использование автоматического порога и работы только с контурами снижает объем вычислений. Учет величины градиента (а не только факта точки) повышает устойчивость к слабоконтрастным границам.

Эффективность полностью зависит от качества выделения контура, что проблематично при шуме, текстурах или плохом освещении. Сканирование изображения окном и построение сигнатур вычислительно затратно. Метод анализирует только форму, игнорируя текстуру и семантический контекст, что ограничивает его на сложных сценах.

Работа представляет собой классический feature-based подход, актуальный до распространения глубокого обучения. Это грамотное инженерное решение для инвариантного распознавания формы. Сегодня такие методы уступили место нейросетям, но сохраняют ценность как фундамент для понимания геометрических принципов компьютерного зрения.

M. Mahlke, E. Solano «The ssos pipeline: Identification of Solar System objects in astronomical images» [10]

Статья представляет программный конвейер ssos для автоматического обнаружения тел Солнечной системы (астероидов, комет, транснептуновых объектов) на астрономических изображениях. Конвейер предназначен для извлечения «скрытых» данных о таких объектах из огромных архивов обзорных и целевых наблюдений, где они были зафиксированы случайно. Основной принцип – анализ серии снимков одного участка неба, сделанных в разное время, для выделения источников с линейным собственным движением, что является ключевым признаком тел Солнечной системы.

Общая архитектура разработанного технологического решения приведена на рисунке 1. Конвейер построен на трех ключевых этапах:

- 1) SExtractor обнаруживает все источники;
- 2) SCAMP астрометрически привязывает снимки и вычисляет движение источников;
- 3) python-фильтр отбирает именно объекты Солнечной системы по ключевым признакам – линейному собственному движению, количеству обнаружений и согласованности их изображений. Это универсальный инструмент, балансирующий между полнотой и чистотой выборки.

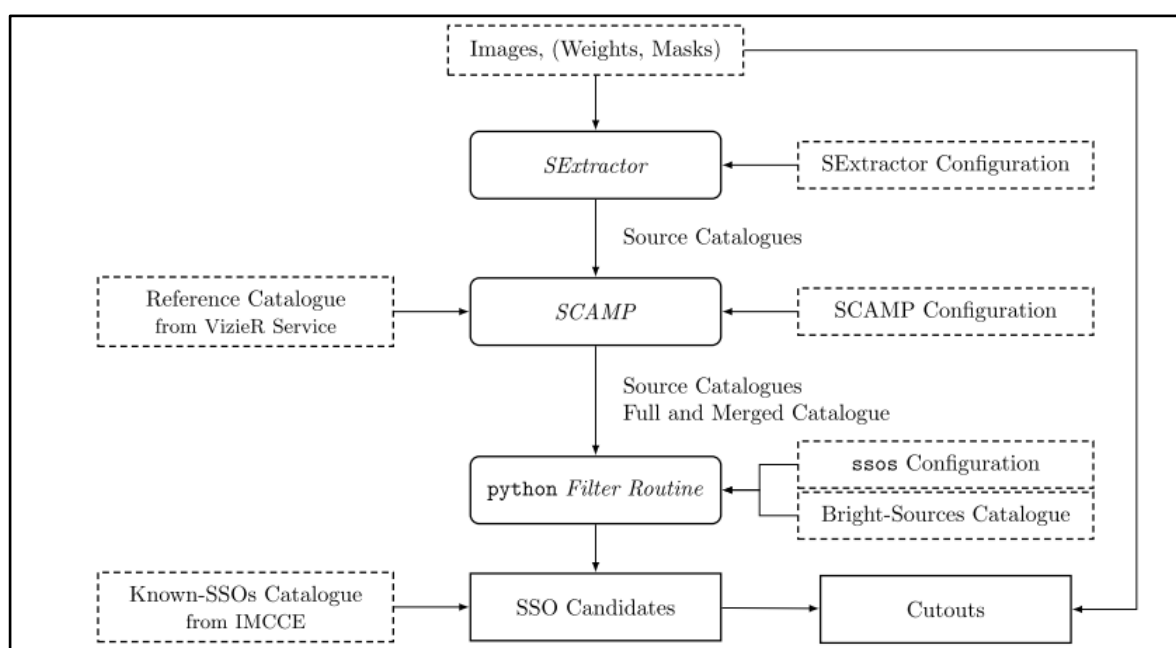


Рисунок 1 – Блок-схема конвейера

Решение практично, универсально и проверено. Оно повышает научную отдачу архивов, работает с данными разных телескопов (от узкополосных до обзорных) и демонстрирует низкий уровень ложных срабатываний (около 2% в оптимальном режиме). Открытый код делает его доступным.

Эффективность конвейера зависит от качества исходных данных и начальной настройки параметров. Он требует значительных вычислительных ресурсов для больших объемов данных и потенциально может пропускать объекты с нелинейным движением или на снимках с большими временными разрывами.

ssos – это не прорывной алгоритм, а эффективное инженерное решение, которое делает рутинную, но важную задачу поиска движущихся объектов доступной для широкого круга обсерваторий, заполняя нишу между мощными специализированными системами и ручным анализом.

Nakamura. K. «Optimizing Combinations of Teaching Image Data for Detecting Objects in Images» [12]

Статья предлагает метод оптимизации обучающей выборки для нейросетей, чтобы уменьшить их зависимость от фона при детекции объектов в условиях фиксированной камеры. Авторы используют динамическое программирование (DP) для отбора изображений, которые максимально сфокусированы на объекте и минимально похожи по фону, что должно улучшить работу модели (Mask R-CNN) в нестабильной среде, например, в комнате, где предметы перемещают

Процесс состоит из двух ключевых частей:

1. Оптимизация данных: создается усредненное «масочное» изображение (mask data) из всех объектов обучающей выборки, схема формирования которого приведена на рисунке 2. Для каждого изображения вычисляются: а) расстояние признаков (используя A-KAZE) до этой маски и б) сходство фонов по цветовой гистограмме. Алгоритм динамического программирования решает задачу нахождения оптимального набора изображе-

ний, который максимизирует суммарное расстояние признаков (т.е. разнообразие ракурсов/состояний объекта) при ограничении на суммарное сходство фонов (т.е. предотвращает доминирование одного типа фона).

2. Обнаружение и отслеживание: отобранные данные обучают Mask R-CNN для детекции. Затем вторая подсистема отслеживает объекты по времени, сравнивая новые обнаружения с историей через анализ цветовых гистограмм областей объектов и их координат, определяя появление, исчезновение и перемещение.

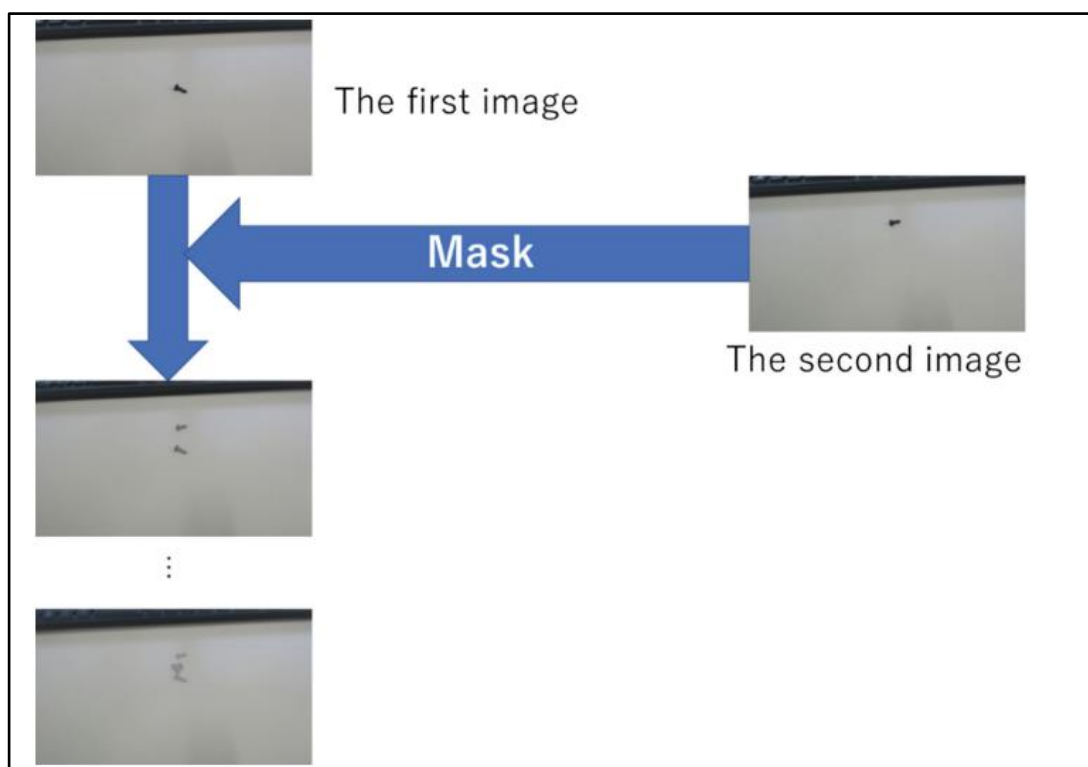


Рисунок 2 – Создание «масочных данных» путем наложения изображений с прозрачностью

Ключевой недостаток – неподтвержденная эффективность: эксперимент не показал преимуществ метода, так как результаты на отобранных данных почти не отличались от обучения на всей выборке. Основной тезис работы не был проверен на сложном фоне. Кроме того, метод трекинга, основанный только на сравнении цветовых гистограмм, ненадежен и может давать сбои при схожей окраске объектов или изменении освещения, что ограничивает практическое применение.

Исследование представляет собой интересное, но незавершенное прикладное предложение. Идея использования динамического программирования для балансировки обучающей выборки между разнообразием объекта и однородностью фона является интригующей. Однако работа осталась на уровне концепции, так как ее центральная гипотеза не получила эмпирического подтверждения в рамках проведенного эксперимента. Метод требует серьезной доработки, валидации на сложных данных (со «сложным» фоном) и усиления модуля трекинга для рассмотрения как практичное решение.

1.2. Анализ работы YOLO

Теоретической и алгоритмической основой для проведения анализа современных систем одноэтапного детектирования объектов послужили фундаментальные исследования и прикладные работы отечественных и зарубежных авторов в области глубокого обучения. В частности, в работе рассматривается и берется за основу систематизированный опыт эволюции нейросетевых архитектур, подробно описанный в трудах М. Е. Васильева, А. С. Шалимова и О. А. Савиной [3], которые детально исследовали структурные изменения модулей признаков в различных версиях алгоритмов. Кроме того, детальный анализ и декомпозиция компонентов Backbone и Neck опираются на классические архитектурные решения создателей моделей – Ч. И. Ванга, А. Бочковского, а также на техническую документацию исследовательских коллективов, заложивших базу для гибкого инференса нейросетей на вычислительных системах с ограниченными ресурсами.

В области компьютерного зрения ключевое место среди одноэтапных (single-stage) подходов занимают алгоритмы семейства YOLO (You Only Look Once). В отличие от двухэтапных (two-stage) архитектур класса R-CNN, они объединяют генерацию регионов и классификацию в рамках единого прохода по сверточной сети. Это минимизирует вычислительные задержки и обеспечивает обработку данных в режиме реального времени.

Одним из наиболее стабильных технологических решений является архитектура YOLOv5, схема которой представлена на рисунке 3. Ее структура включает три компонента: основную сверточную сеть (Backbone), промежуточный блок агрегации признаков (Neck) и прогнозирующую «голову» (Head). Выделение пространственных признаков раstra осуществляется за счет модулей C3 и пространственного пирамидального пулинга (SPP). Блок Neck выполняет каскадное объединение разномасштабных признаков, а Head генерирует финальные векторы с координатами ограничивающих рамок (bounding boxes) и вероятностями классов.

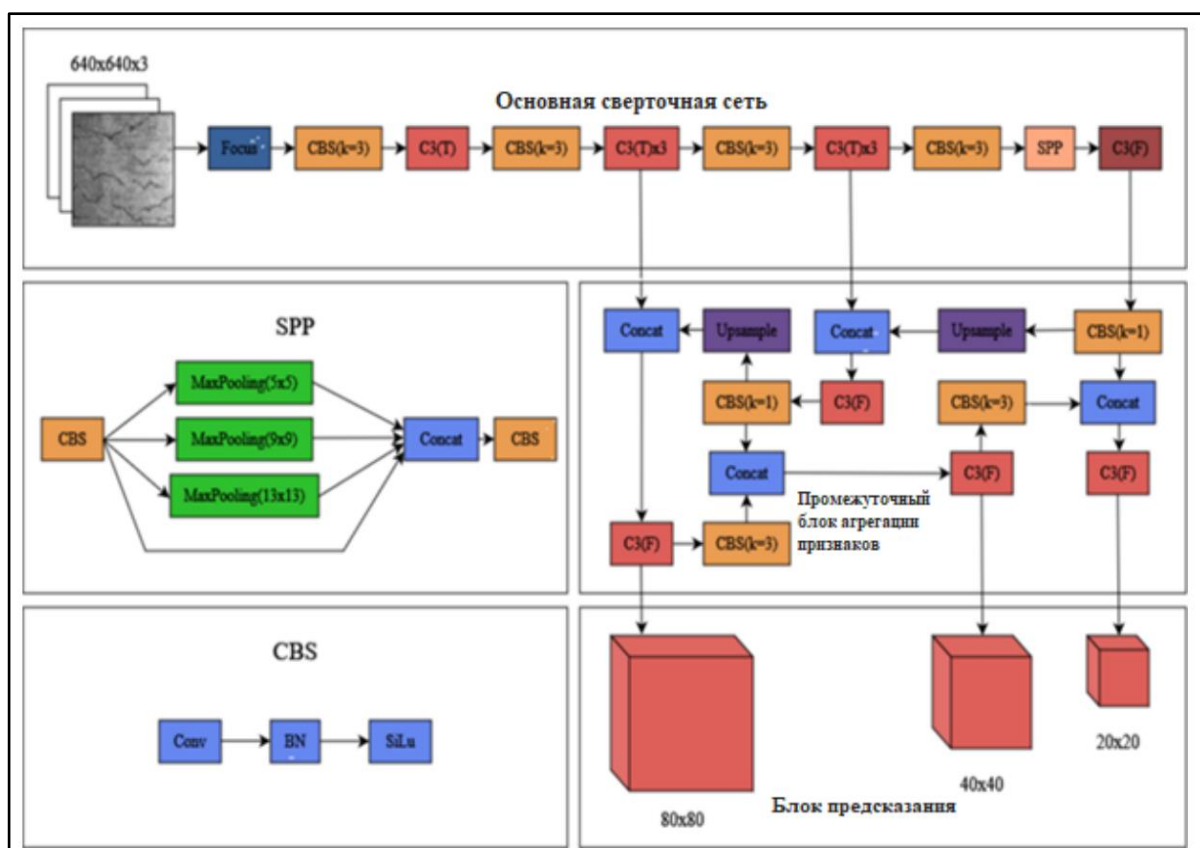


Рисунок 3 – Схема базовой архитектуры модели YOLOv5

Эволюция одноэтапных детекторов нашла отражение в архитектуре YOLOv7, структурная схема которой приведена на рисунке 4. Модификация оптимизирует вычислительные ресурсы за счет интеграции следующих компонентов:

1. Блоки CBS: модули, сочетающие свертку, пакетную нормализацию и активацию Swish.

2. Модули MP: операции Max Pooling для снижения пространственного разрешения карт признаков.
3. Сети ELAN-H: механизмы эффективной агрегации слоев для точного мультимасштабного распознавания объектов.
4. Модули SPPCSPC: комбинация пирамидального пулинга и архитектуры CSPNet, сокращающая число параметров и стабилизирующая сходимость модели.

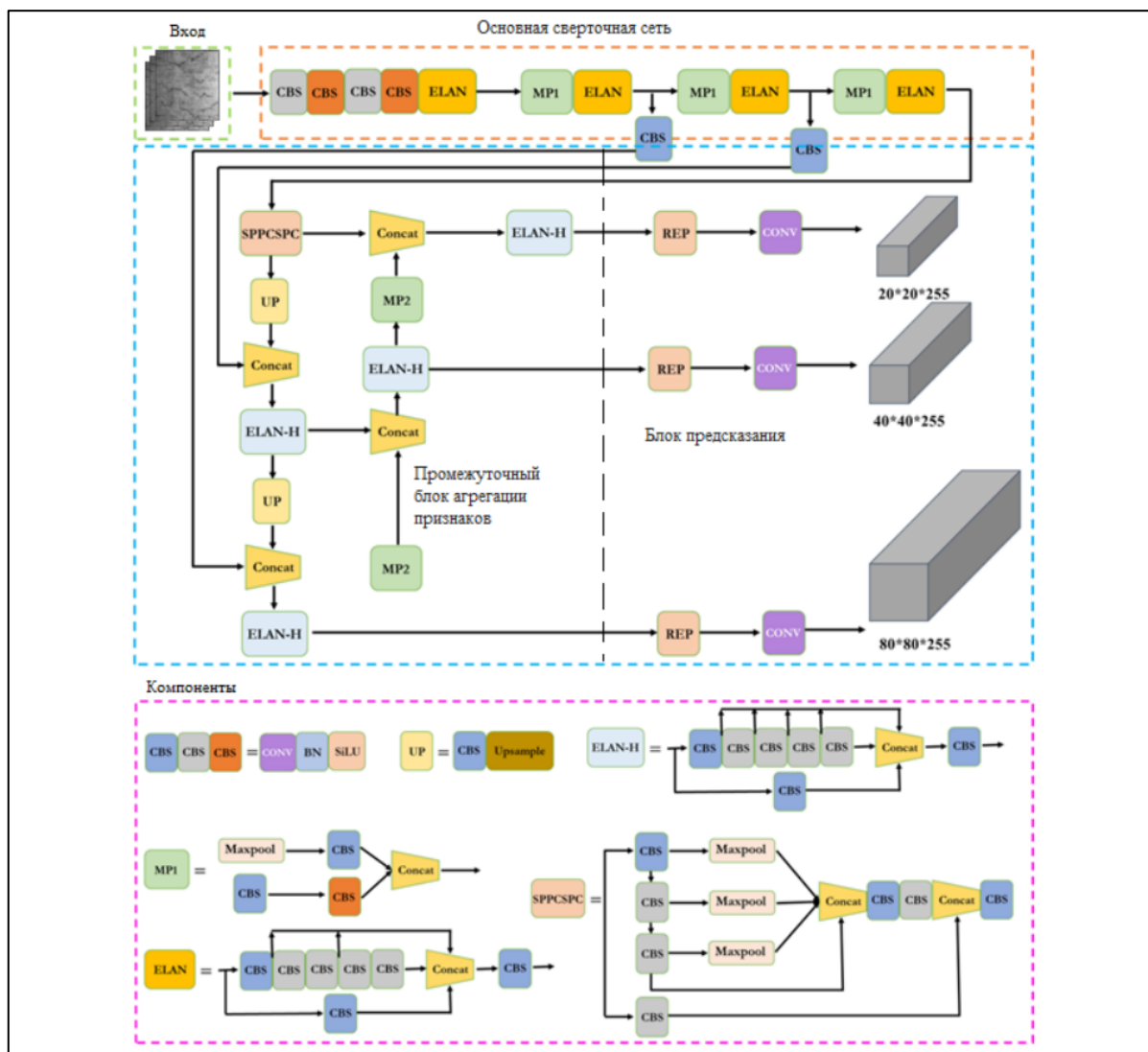


Рисунок 4 – Архитектурная структура модели детекции YOLOv7

Наиболее прогрессивной и сбалансированной парадигмой в семействе на текущий момент выступает архитектура YOLOv11, модульная структура которой представлена на рисунке 5. Сохраняя классическое трехкомпонентное разделение, она внедряет принципиально новые подходы к извлечению

детальных признаков. Главной инновацией Backbone-слоя в YOLOv11 является блок C3K2, представляющий собой эволюцию CSP-архитектуры. В отличие от предшественников (например, модуля C2F), блок C3K2 последовательно обрабатывает информационный поток через серию мелких сверточных ядер (размером 3 на 3) и каскад модулей Bottle Neck, что позволяет существенно снизить общую ресурсоемкость и повысить энергоэффективность инференса на маломощных ЭВМ.

В промежуточном блоке Neck применяется быстрый пространственный пирамидальный пулинг (SPFF) для контекстного извлечения данных. Важным дополнением стал механизм пространственного внимания C2PSA с параллельными PSA-модулями, позволяющий сети динамически фокусироваться на информативных областях, изолируя перекрытые или размытые объекты.

Прогнозирующая «голова» YOLOv11 реализует мультиуровневый подход к предсказаниям, одновременно обрабатывая карты признаков с трех независимых уровней детализации (P3, P4, P5), что обеспечивает сквозную детекцию разномасштабных элементов.

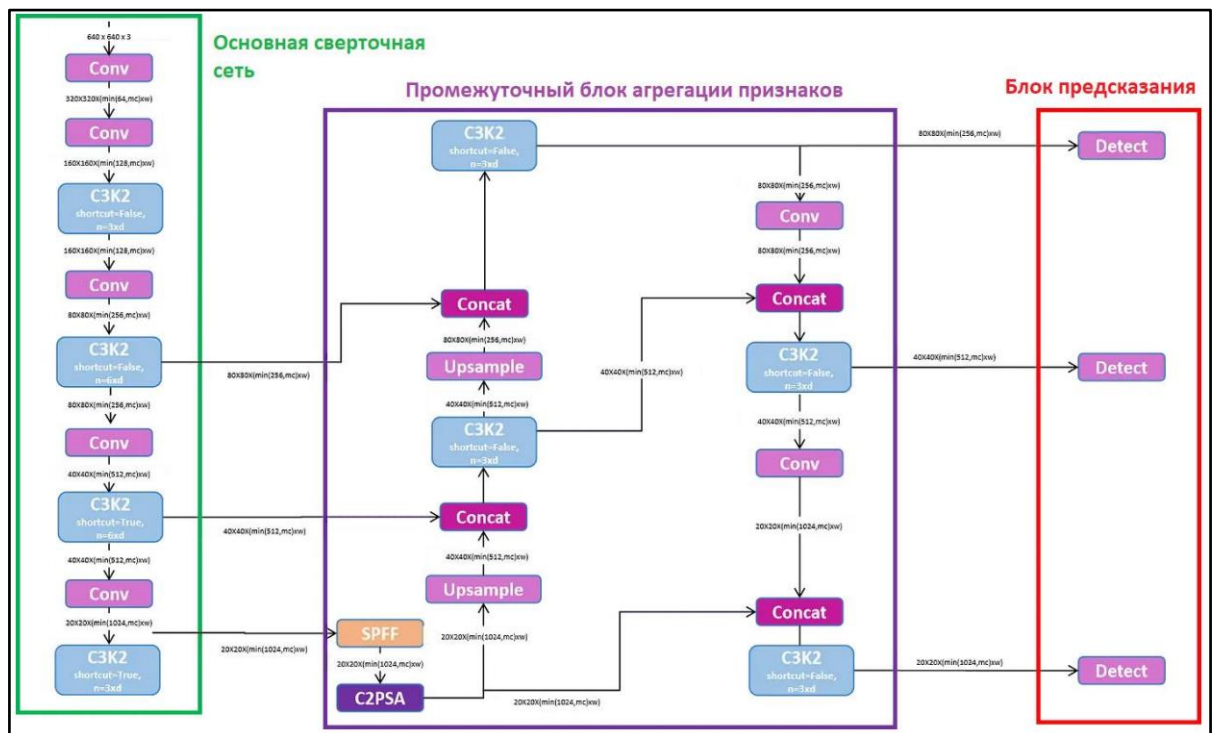


Рисунок 5 – Модульная архитектура нейросети YOLOv11

В качестве заключения можно сказать, что экспериментальные исследования и логи обучения моделей (на примере верификации дефектов поверхностей) показывают, что YOLOv7 обладает высокой стабильностью и достигает средних значений точности mAP50 на уровне 0.990 к десятой эпохе обучения, однако требует значительных временных затрат на одну итерацию. Архитектура YOLOv11 демонстрирует несколько более сложный процесс сходимости на ранних эпохах, обусловленный комплексной структурой блоков внимания, но в процессе стабилизации градиентов достигает превосходящих метрик точности ($mAP50 = 0.992$) при высокой энергоэффективности и гибкости, что делает ее наиболее перспективным ядром для разработки настольных интеллектуальных платформ.

1.3. Анализ работы OCR

При исследовании математических методов и построении сквозного конвейера оптического распознавания символов (OCR) за научно-методологическую основу была принята концептуальная модель, разработанная исследовательской группой под руководством Т. В. Хоменко, А. А. Иргалиева и В. Д. Тараканова [16]. Их подход к трехэтапной декомпозиции процесса ОРС (обработка, сегментация, распознавание) позволил формализовать требования к качеству входных растровых данных. Математическое описание процессов фильтрации и бинаризации матриц базируется на алгоритмических наработках В. Б. Стецюка, В. Л. Никитенкова и А. А. Поберия, а в качестве концептуальной базы для проектирования классифицирующих слоев искусственной нейронной сети были взяты структурные схемы полносвязных конфигураций, предложенные А. В. Навойчиком и Н. И. Гуриным.

Вторым фундаментальным компонентом систем автоматизированного анализа графических документов выступает подсистема оптического распознавания символов (Optical Character Recognition, OCR). Процесс ОРС представляет собой сквозной конвейер обработки информации, направленный на трансформацию неструктурированного растрового изображения в

цифровой машиночитаемый текстовый формат. На основе системного анализа выделяются три последовательных этапа реализации ОРС-систем, структура которых приведена на рисунке 6: предварительная обработка графических данных, многоуровневая сегментация и непосредственное распознавание (классификация) символьных объектов.

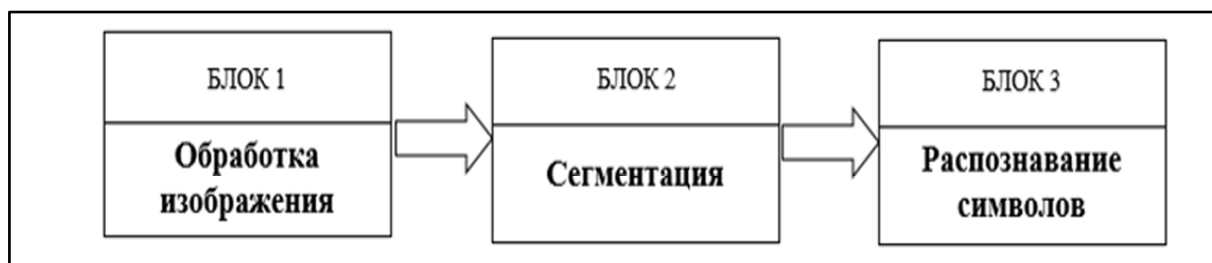


Рисунок 6 – Схема этапов процесса распознавания символов

Этап 1. Предварительная инженерная обработка раstra

Входные графические данные (сканированные копии, фотографии) часто характеризуются наличием геометрических перекосов и высокочастотных шумов, существенно снижающих точность анализа. Для устранения данных факторов в системе последовательно реализуются следующие алгоритмические процессы:

1. Выравнивание (Deskewing): устранение угла наклона документа посредством автоматического выявления доминирующих горизонтальных векторов строк текста и принудительного разворота матрицы изображения.

2. Фильтрация шумов (Blurring): для подавления аддитивного гауссовского шума применяется двумерная фильтрация Гаусса. Математически преобразование каждого пикселя в двух измерениях (x, y) описывается функцией распределения (формула 1):

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (1)$$

где σ – среднеквадратическое отклонение нормального распределения.

3. Двухэтапная бинаризация: перевод исходного цветного пространства RGB в бинарный черно-белый массив. На первом шаге вычисляется матрица градаций серого по формуле интенсивности (формула 2):

$$g = \frac{(66 \cdot R + 129 \cdot G + 25 \cdot B + 128)}{256} + 16 \quad (2)$$

на втором шаге для точного выделения контуров границ текстовых областей применяется оператор Кенни.

4. Морфологическая зачистка: удаление из бинарной матрицы посторонних геометрических линий, не принадлежащих к структуре текста, методами морфологической обработки и алгоритма Блумберга.

Этап 2. Алгоритмы многоуровневой сегментации

Целью сегментации является декомпозиция очищенного бинарного изображения на изолированные матрицы строк, слов и отдельных символов. Для этого строится матрица яркости $C = \{c_{ij}\}$ размерностью $n \times m$ (где n – ширина, а m – высота кадра. Процесс разделения включает три иерархических шага.

1. Сегментация строк: вычисляется вектор средних значений яркости для каждой горизонтальной строки пикселей (формула 3):

$$f_{j(c)} = \frac{1}{n} \sum_{i=1}^1 c_{ij} \quad (3)$$

границы строк (верхняя f_{top} и нижняя f_{bott} определяются через локальные минимумы относительно средней яркости всего изображения

2. Сегментация слов: выделенные строки анализируются вертикально по столбцам для поиска межсловных интервалов с использованием вектора средних яркостей столбцов (формула 4):

$$f_{i(c)} = \frac{1}{m} \sum_{j=1}^m c_{ij} . \quad (4)$$

3. Сегментация символов: наиболее сложная процедура, базирующаяся на поиске локальных минимумов яркости в скользящем интервале, размер которого жестко коррелирует с высотой строки ($h_j = 0.3 \cdot t$). Ошибочные (ложные) границы, возникающие внутри широких слабосвязанных букв (например, «Ц», «П», «Н»), принудительно исключаются путем разделения символа на три вертикальных уровня (30%, 45% и 60% от общей высоты) и их независимого анализа на связность с соседними столбцами.

Этап 3. Классификация и распознавание объектов

Финальный этап ОРС обеспечивает сопоставление сегментированных матриц конкретных буквенных знаков со стандартными символами целевого алфавита. Для решения данной задачи классификации в современных изолированных архитектурах применяются искусственные нейронные сети, концептуальная схема, одной из которых представлена на рисунке 7

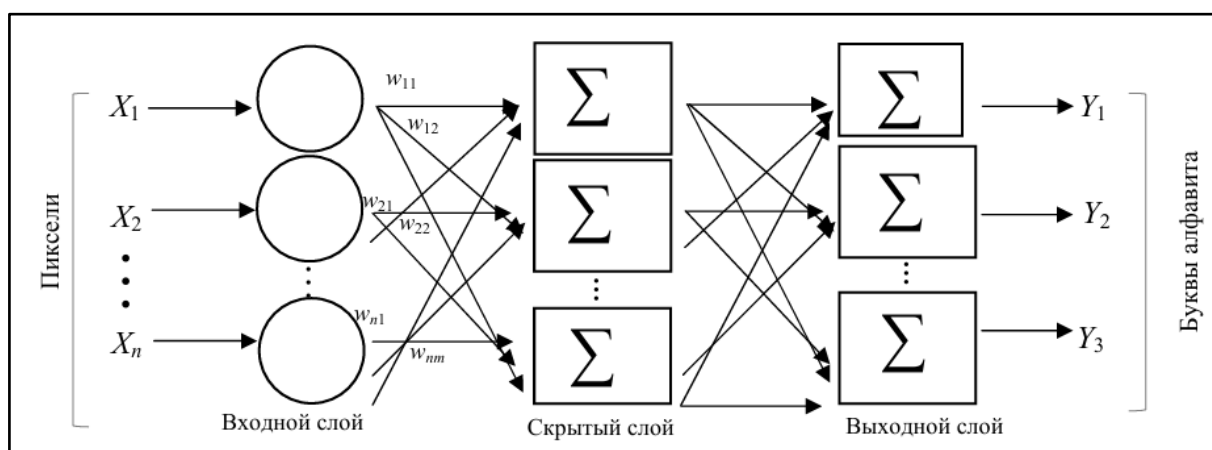


Рисунок 7 – Схема полносвязной нейронной сети: X – входные данные; Y – выходные данные

Матрица пикселей нормализованного изображения символа (например, размерностью (28×28)) подается на входной слой нейросети, содержащий $(n \times t)$ нейронов. Выходной слой сети состоит из k нейронов, количество которых строго эквивалентно размерности используемого алфавита (включая латинские, кириллические символы и цифровые значения). Индуцированное локальное поле u (активационный сумматор) для каждого

последующего скрытого и выходного слоя рассчитывается на основе весовых коэффициентов связей w_{ij} по математической модели (формула 5):

$$y = f(u), u = \sum_{i=1}^n w_{ij}x_i + w_0x_0 \quad (5)$$

где x_i – входные сигналы (значения пикселей), а w_0x_0 – инициализирующее смещение нейрона. Внедрение современных OCR-модулей (таких как EasyOCR), совмещающих предварительную сверточную обработку и рекуррентные слои анализа контекста, позволяет минимизировать ошибки классификации схожих по начертанию символов (например, буквы «о» и цифры «0») за счет постобработки и лингвистической проверки цепочек символов на корректность.

Таким образом, проведенный детальный анализ математических методов и алгоритмических конвейеров оптического распознавания символов позволяет сделать вывод о том, что точность извлечения текстовой информации напрямую зависит от качества реализации каждого из трех изолированных этапов: предварительной фильтрации, многоуровневой пространственной сегментации строк и символов, а также финальной нейросетевой классификации.

Изучение подходов исследовательской группы Т. В. Хоменко продемонстрировало, что ключевой уязвимостью классических систем ОРС является этап жесткой сегментации, при котором начертания нестандартных или близко расположенных букв могут приводить к ложному дроблению символьных матриц.

1.4. Анализ аналогичных проектов

Разработка приложения для идентификации объектов на изображениях ведется в контексте существующих решений, которые можно разделить на несколько категорий: облачные веб-сервисы, мобильные приложения и научно-исследовательские фреймворки. Анализ ключевых аналогов в каждой категории позволяет выявить их сильные и слабые стороны, что необходимо для определения целевой ниши и уникальных особенностей разрабатываемого приложения.

Веб-сервис «Roboflow» [5]

Roboflow – это комплексная облачная платформа, предназначенная для полного цикла разработки компьютерного зрения. Она предоставляет инструменты для сбора, аннотирования, предобработки и разметки датасетов, а также для обучения и развертывания моделей детекции объектов.

Платформа Roboflow предлагает комплексный набор инструментов для управления всем жизненным циклом проекта компьютерного зрения. Центральным элементом является удобный веб-интерфейс для аннотирования изображений и видео, который поддерживает командную работу с разграничением прав доступа и предоставляет мощные инструменты разметки, включая предварительную автоматизацию с помощью модели SAM (Segment Anything Model) от Meta для ускорения создания масок. Для повышения качества и разнообразия данных система включает обширный модуль автоматической предобработки и аугментации, позволяющий применять стандартные методы (изменение яркости, контраста, повороты, кадрирование) и продвинутые техники, такие как мозаика (mosaic) и случайное панорамирование (random perspective), которые особенно эффективны для обучения современных детекторов объектов. После завершения разметки Roboflow позволяет одним кликом сгенерировать готовый к обучению датасет: платформа автоматически конвертирует аннотации в выбранный формат (COCO JSON, YOLO Darknet/TXT, TensorFlow TFRecord, Pascal VOC XML и другие), создает конфигурационные файлы и структурирует данные

для немедленного использования в популярных фреймворках машинного обучения.

Для этапа обучения Roboflow предоставляет собственный облачный тренировочный кластер (Roboflow Train), который устраняет необходимость в локальных вычислительных ресурсах. Пользователь может выбрать из множества предварительно настроенных архитектур, включая семейства YOLO (YOLOv5, YOLOv8, YOLOv11) и EfficientDet, запустить обучение с помощью нескольких кликов и отслеживать метрики в реальном времени через тот же веб-интерфейс. После успешного обучения модель переходит на стадию развертывания (Deploy). Roboflow предлагает несколько вариантов: модель можно опубликовать как масштабируемый REST API-эндпоинт для простой интеграции в веб- или мобильные приложения, либо экспортировать ее в различных форматах (ONNX, TensorFlow Lite, TensorRT, Core ML) для скачивания и локального использования на edge-устройствах, таких как NVIDIA Jetson, Raspberry Pi или мобильные телефоны. Весь этот конвейер – от исходных изображений до работающей модели – интегрирован в единую платформу, значительно ускоряя и упрощая разработку проектов компьютерного зрения.

Преимущества: all-in-one решение, охватывает весь конвейер – от данных до работающей модели; низкий порог входа; коллаборация, позволяющая реализовывать командные проекты; активное сообщество и обширная документация.

Недостатки: ограниченность бесплатного режима; работа сосредоточена в экосистеме Roboflow; ограниченная кастомизация моделей; зависимость от интернет-соединения и серверов компании.

Вывод о продукте: Roboflow является мощным промышленным стандартом для команд, которым нужно быстро создать рабочий прототип или проект. Однако для образовательных целей, глубокой кастомизации алгоритмов или работы в условиях ограниченного или отсутствующего интернета требуется локальное решение.

Мобильное приложение «Google Lens» [11]

Google Lens – это кроссплатформенное мобильное приложение, использующее технологии компьютерного зрения и машинного обучения для анализа изображений в реальном времени через камеру смартфона или из галереи.

Google Lens – это инструмент визуального поиска и идентификации, встроенный в экосистему Android и Google Фото. Он способен мгновенно распознавать и анализировать широкий спектр объектов в реальном времени через камеру или на загруженных изображениях, включая растения, породы животных, архитектурные достопримечательности, обложки книг, различные товары, а также считывать штрих-коды и тексты. Его ключевой функцией является прямой поиск по изображению, который интегрирован с ядром поисковой системы Google для нахождения похожих товаров в онлайн-магазинах, получения справочной информации, рецензий или новостей об обнаруженном объекте.

Особой интеллектуальной возможностью является функция «Умный текст» (Smart Text), которая не просто копирует распознанный с изображения текст, но и позволяет совершать с ним контекстные действия: например, позвонить по обнаруженному номеру телефона, сохранить контакт в адресную книгу, открыть веб-ссылки в браузере или скопировать адрес для навигации. Для образовательных целей предусмотрен специализированный режим «Домашнее задание», который может распознавать как печатные, так и рукописные математические уравнения, демонстрируя не только готовый ответ, но и пошаговое объяснение решения, что делает его мощным вспомогательным инструментом для учебы.

Все эти функции глубоко интегрированы в операционную систему Android, что позволяет легко вызывать Lens из приложения камеры, Google Фото, Google Ассистента или через виджет поиска. Эта интеграция обеспечивает бесшовный пользовательский опыт, где распознанные объекты, тек-

сты или действия становятся частью привычного рабочего процесса, связывая физический мир с цифровыми возможностями поиска и автоматизации Google.

Преимущества: удобство и скорость; мультизадачность; высокая точность; бесплатный продукт.

Недостатки: отсутствие API для кастомной детекции; закрытая архитектура; зависимость от интернет-соединения и серверов компании.

Вывод о продукте: Google Lens – это эталон пользовательского опыта в массовом потребительском сегменте. Однако оно абсолютно непригодно для научных, промышленных или специализированных задач, где требуется контроль над процессом, работа с узкими доменами и интеграция в собственные системы.

Научно-исследовательский фреймворк для ПК «Detectron2» [4]

Detectron2 – это фреймворк от Facebook AI Research (FAIR), написанный на PyTorch. Он предоставляет реализацию state-of-the-art алгоритмов для задач детекции и сегментации изображений и предназначен в первую очередь для исследователей и инженеров.

Detectron2 представляет собой фреймворк для задач компьютерного зрения от Facebook AI Research (FAIR), созданный на базе PyTorch. Его фундаментом является библиотека современных, готовых к использованию и высокооптимизированных реализаций ключевых архитектур детекции, сегментации и поз человека, таких как Faster R-CNN, Mask R-CNN, RetinaNet и DensePose. Одним из главных преимуществ фреймворка является высокая гибкость и модульность: он построен по принципу конструктора, что позволяет исследователям и инженерам легко заменять отдельные компоненты, такие как бэкбоны (ResNet, Swin Transformer), головы детекции или функции потерь, для проведения экспериментов и создания кастомных моделей под специфические задачи. Для эффективной работы с вычислительными

ресурсами фреймворк поддерживает распределенное обучение на нескольких GPU из коробки, что ускоряет процесс тренировки моделей на больших датасетах.

Фреймворк Detectron2 содержит развитую инфраструктуру данных со встроенной поддержкой популярных датасетов (COCO, Pascal VOC, Cityscapes) через DatasetCatalog и возможностью интеграции пользовательских наборов. Оптимизация для высокопроизводительного обучения и инференса делает его применимым как для быстрого прототипирования, так и в продакшн-сценариях. Развитая кодовая база и активное инженерное сообщество обеспечивают стабильную поддержку и развитие проекта.

Преимущества: максимальный уровень контроля над всеми аспектами модели и процесса обучения; высокая точность.

Недостатки: высокий порог входа; отсутствие графического интерфейса; ресурс зависимый.

Вывод о продукте: Detectron2 – это профессиональный инструмент для создания решений в области компьютерного зрения. Он представляет собой противоположный полюс по сравнению с Google Lens: максимальная мощь и гибкость в ущерб простоте и доступности.

Выводы по первой главе

В первой главе выполнен детальный анализ предметной области, в ходе которого на основе сравнения аналогов (Roboflow, Google Lens, Detectron2) были выявлены их ключевые ограничения: зависимость от интернет-соединения и отсутствие локальной кастомизации. Рассмотренная эволюция одноэтапных детекторов (YOLOv5, YOLOv7) и алгоритмов OCR позволила обосновать выбор наиболее прогрессивных подходов, что послужило базой для разработки автономного десктопного приложения «Vision Pro AI» на стыке актуальной архитектуры YOLOv11 и библиотеки EasyOCR.

2. ПРОЕКТИРОВАНИЕ

Важнейшим этапом жизненного цикла разработки программного обеспечения является детальное проектирование, направленное на формализацию условий функционирования системы, определение ее внутренних и внешних интерфейсов, а также формирование структуры обрабатываемых данных. На основе детального анализа предметной области компьютерного зрения и оптического распознавания символов (OCR) формируется комплекс критериев, определяющих состав и логику работы разрабатываемого настольного приложения.

2.1. Требования к системе

При разработке десктопного приложения сформулирована совокупность требований, которым должна соответствовать итоговая система. Все требования разделены на два типа: функциональные и нефункциональные.

Функциональные требования

Функциональные требования определяют конкретное поведение программной системы, перечень решаемых прикладных задач, доступные пользователю сценарии управления, а также закладывают алгоритмическую основу реакции приложения на внешние управляющие воздействия и информационные потоки.

На основе детальной декомпозиции бизнес-логики процессов детекции объектов, распознавания печатных символов и генерации экспертной отчетности были сформулированы и зафиксированы следующие функциональные требования к настольному приложению.

1. Система должна предоставлять пользователю возможность запускать графический интерфейс приложения и корректно закрывать его вместе со всеми фоновыми процессами.

2. Система должна предоставлять пользователю возможность выбирать исходное изображение с компьютера (в форматах JPEG, PNG) для последующего анализа.

3. Система должна предоставлять пользователю возможность сохранять выбранные файлы во внутреннюю папку приложения с созданием уникального имени (таймстампа), чтобы избежать потери путей.
4. Система должна предоставлять пользователю возможность выбирать активную ИИ-модель (YOLO) из выпадающего списка на главном экране.
5. Система должна предоставлять пользователю возможность настраивать порог точности обнаружения объектов с помощью графического ползунка (слайдера).
6. Система должна предоставлять пользователю возможность включать режим «Только текст» с помощью чекбокса, чтобы распознавать только надписи без поиска объектов.
7. Система должна предоставлять пользователю возможность запускать комплексный анализ кадра, включающий одновременный поиск объектов нейросетью YOLOv11 и распознавание текста через EasyOCR.
8. Система должна предоставлять пользователю возможность просматривать результаты анализа в специальном окне-редакторе в виде списка элементов с иконками типов «Объект» или «Текст».
9. Система должна предоставлять пользователю возможность управлять элементами в редакторе с помощью чекбоксов и кнопок быстрого выбора («Выделить все» / «Снять все»).
10. Система должна предоставлять пользователю возможность автоматически вырезать и сохранять фрагменты выбранных объектов в отдельные папки для создания собственного датасета.
11. Система должна предоставлять пользователю возможность сохранять итоговые результаты анализа (дата, имя файла, найденные объекты, текст и заметки) в локальную базу данных SQLite.
12. Система должна предоставлять пользователю возможность просматривать архив прошлых исследований на экране истории в виде графических карточек-протоколов.

13. Система должна предоставлять пользователю возможность выполнять быстрый «живой поиск» по названию и фильтровать историю по временным периодам («Сегодня», «За неделю», «Все время»).

14. Система должна предоставлять пользователю возможность удалять выбранные протоколы исследований из базы данных с сохранением связанных медиаданных в локальном изолированном хранилище для целей последующего формирования расширенных обучающих выборок (датасетов).

15. Система должна предоставлять пользователю возможность просматривать общую статистику работы приложения (общее количество тестов, среднее число объектов) в виде графиков.

16. Система должна предоставлять пользователю возможность выгружать всю накопленную историю исследований в сводную таблицу формата Excel.

17. Система должна предоставлять пользователю возможность генерировать и сохранять официальный многостраничный PDF-отчет с водяным знаком и внесенными текстовыми заметками.

18. Система должна предоставлять пользователю возможность открывать встроенный менеджер моделей для фонового скачивания весов нейросетей, добавления кастомных моделей или их удаления.

Нефункциональные требования

Нефункциональные требования задают свойства системы и рамки, в которых ведется разработка. Они определяют способы реализации функций и устанавливают критерии качества системы. Сформулированы следующие нефункциональные требования.

1. Система должна быть разработана на базе объектно-ориентированного фреймворка PyQt6 для обеспечения кроссплатформенности и работы на операционных системах семейства Windows.

2. Система должна разделять кодовую базу на три независимых слоя: интерфейс (UI), логику и нейросети (Engine), работу с базой данных (Data), чтобы код оставался понятным и модульным.

3. Система должна выполнять все длительные операции (инференс нейросетей, генерация PDF, скачивание моделей из сети) в фоновых потоках QThread, чтобы графический интерфейс не зависал во время работы.

4. Система должна автоматически рассчитывать пути к файлам относительно папки запуска приложения, обеспечивая переносимость программы на любой другой компьютер без изменения кода.

5. Система должна стабильно работать и автоматически переключать вычисления нейросетей на центральный процессор (CPU), если на компьютере пользователя отсутствует совместимая видеокарта с поддержкой CUDA.

6. Система должна поддерживать единый стиль оформления интерфейса (дизайн-система Dark Lavender) с возможностью мгновенного переключения цветовой темы для всех окон и графиков.

7. Система должна использовать локальную СУБД SQLite, которая не требует установки и настройки дополнительных серверов баз данных на компьютере пользователя.

2.2. Варианты использования

Для систематизации функциональных требований и описания взаимодействия пользователей с системой построена диаграмма вариантов использования, представленная на рисунке 8.

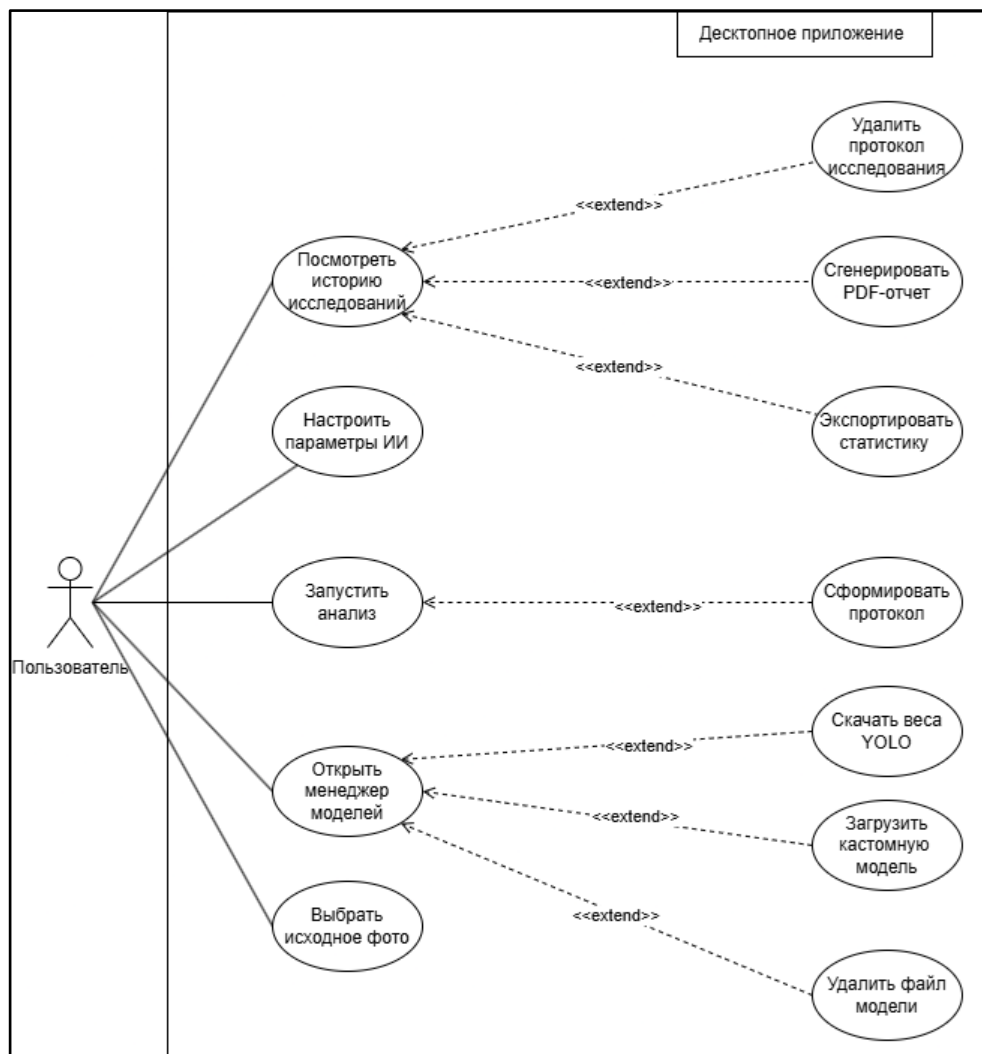


Рисунок 8 Диаграмма вариантов использования

В разрабатываемой системе выделяется один основной актер – Пользователь, выступающий в роли оператора программного комплекса. Он взаимодействует с локальной графической оболочкой, при этом ему доступен полный набор функций десктопного приложения без разграничения прав доступа. Оператор может гибко управлять параметрами искусственного интеллекта, выбирая активную ИИ-модель из локального каталога, настраивая порог точности распознавания и активируя режим распознавания текста.

Перед запуском комплексного анализа система позволяет выбрать медиа-файл на диске и выполняет его автоматическое кэширование. Наличие интерактивных элементов интерфейса позволяет оператору адаптировать чувствительность нейросети под специфику конкретного входного раstra, сглаживая дефекты освещения или шумы исходного кадра.

В части работы с результатами распознавания оператор осуществляет визуальную верификацию найденных объектов, настраивает состав отображаемых элементов и подтверждает формирование локального протокола исследования. В этот момент приложение автоматически нарезает графические фрагменты (кропы) обнаруженных объектов для формирования дата-сета. Полученная разметка служит основой для последующего дообучения моделей на специфических выборках непосредственно внутри программы.

Помимо аналитического модуля, пользователю доступен встроенный менеджер моделей. Он предназначен для асинхронного скачивания весов YOLO с удаленного репозитория Hugging Face в фоновом потоке, скачивания кастомных файлов конфигураций и удаления неиспользуемых моделей с диска для экономии места. Для управления накопленной информацией предусмотрен архив исследований с функцией «живой» фильтрации записей в базе данных SQLite непосредственно при вводе текста. Из этого интерфейса пользователь может безвозвратно удалять старые протоколы, а также запускать потоковую генерацию многостраничных PDF-отчетов и экспорт сводной статистики в формат Excel. Автоматизированная выгрузка отчетных документов позволяет бесшовно интегрировать результаты ИИ-анализа в существующие бизнес-процессы предприятия.

Полная спецификация перечисленных вариантов использования приведена в таблицах 1-12 приложения А.

2.3. Архитектура приложения

Проектирование архитектуры разрабатываемого десктопного приложения выполнялось на основе декомпозиции системы на независимые подсистемы и компоненты. Такой подход позволяет обеспечить модульность исходного кода, упростить независимую отладку отдельных частей приложения и реализовать слабую связанность между графической оболочкой и вычислительным ИИ-движком. Все ресурсоемкие операции, такие как инференс нейросетей, скачивание файлов весов с удаленных серверов или работа с дисковой памятью, изолированы от главного потока интерфейса. Диаграмма компонентов системы представлена на рисунке 9.

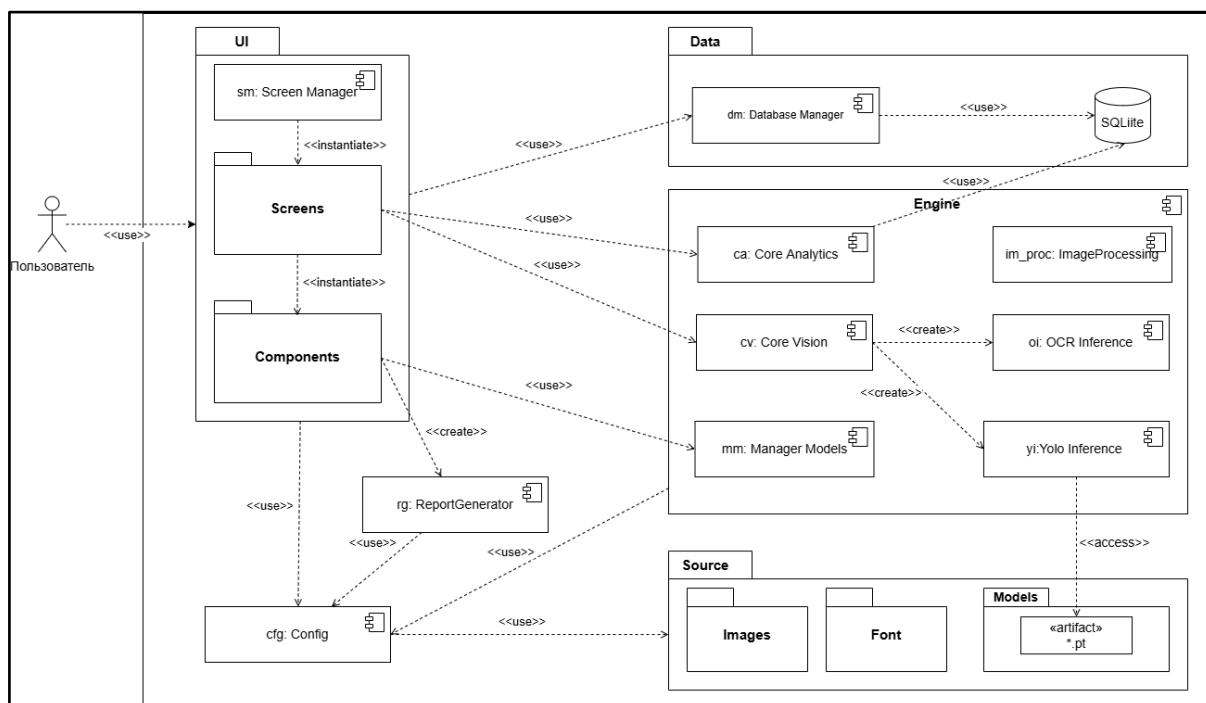


Рисунок 9 – Диаграмма компонентов системы

В соответствии с разработанной диаграммой компонентов, в архитектуре системы выделяется один внешний актер – Пользователь, под которым понимается оператор программного комплекса, взаимодействующий с локальной графической оболочкой. Внутренняя структура приложения распределена по четырем изолированным пакетам, первым из которых является подсистема интерфейса (UI). Внутри нее компонент Менеджер экранов управляет состоянием оконной системы и переключением пространств.

Компонент Экраны интерфейса реализует виджеты страниц, таблицы стилей QSS и механизмы фильтрации архива, а Диалоговые окна отвечают за вывод модальных окон истории исследований и запуск редактора разметки.

Вторым ключевым пакетом системы является вычислительный ИИ-движок, изолированный от графического контекста PyQt6 и оперирующий тензорами. Центральный компонент – Оркестратор ИИ – принимает асинхронные команды из UI, запускает измерительный конвейер и агрегирует метаданные. Непосредственный инференс распределен между двумя подмодулями: компонентом Инференс YOLO (локализация границ объектов нейросетью YOLOv11) и компонентом Инференс OCR (оптическое распознавание символов на базе EasyOCR).

Третьим элементом структуры выступает Служебный слой, обеспечивающий жизненный цикл статических данных приложения. Входящий в него компонент Менеджер моделей работает в фоновом потоке, отвечая за скачивание и удаление файлов весов нейросетей напрямую из репозитория Hugging Face Hub. Компонент Процессор растров выполняет предварительную графическую обработку, кэширует исходные изображения для защиты путей в БД и автоматически нарезает подтвержденные объекты на кропы для формирования локальных датасетов.

Завершающим элементом архитектуры является пакет данных и отчетности. Входящий в него компонент Менеджер БД скрывает SQL-логику, выполняя транзакции к SQLite и возвращая интерфейсу данные в виде коллекций Python. Компонент Аналитика и KPI рассчитывает метрики эффективности библиотекой pandas и строит графики Matplotlib. Компонент Генератор PDF отчетов автоматически верстает экспертные протоколы средствами ReportLab.

Взаимодействие пакетов построено на однонаправленных зависимостях, что позволяет менять логику ИИ-движка независимо от UI. Обмен данными между ними реализован асинхронно через сигналы и слоты

2.4. Диаграммы деятельности и последовательности

Диаграмма деятельности описывает действия, которые выполняет пользователь в процессе инициализации и верификации результатов интеллектуального анализа изображений в системе, включая этапы нейросетевого инференса моделей YOLO и OCR, логические проверки и переходы между состояниями, а также синхронное взаимодействие модулей приложения с файловым кэшем. Пользователь выбирает изображение, а UI загружает его в кэш, запускает потоки, блокирует элементы окна и выводит прогресс-бар. Параллельно в Engine запускается Vision Engine, где по условию выполняется либо последовательный инференс YOLO и OCR, либо только OCR. После этого результаты агрегируются, и сигнал передается в главный поток. В завершение UI скрывает прогресс-бар, разблокирует интерфейс и выдает итоговое изображение. На рисунке 10 представлена диаграмма деятельности, моделирующая данный прецедент «Запустить анализ».

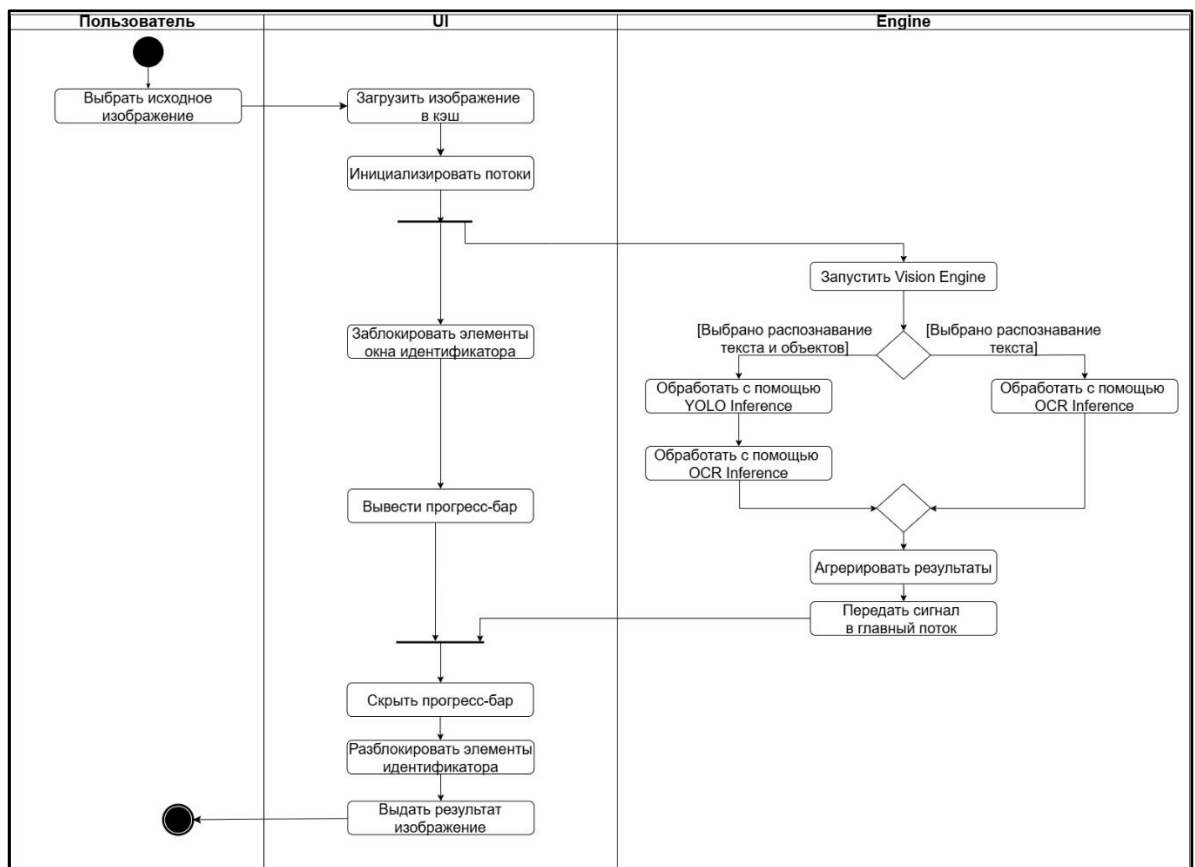


Рисунок 10 – Диаграмма деятельности запустить анализ

Для отображения взаимодействия между пользователем, интерфейсной частью приложения и локальной базой данных при работе с архивом выполненных исследований была разработана диаграмма последовательности. Она представлена на рисунке 11.

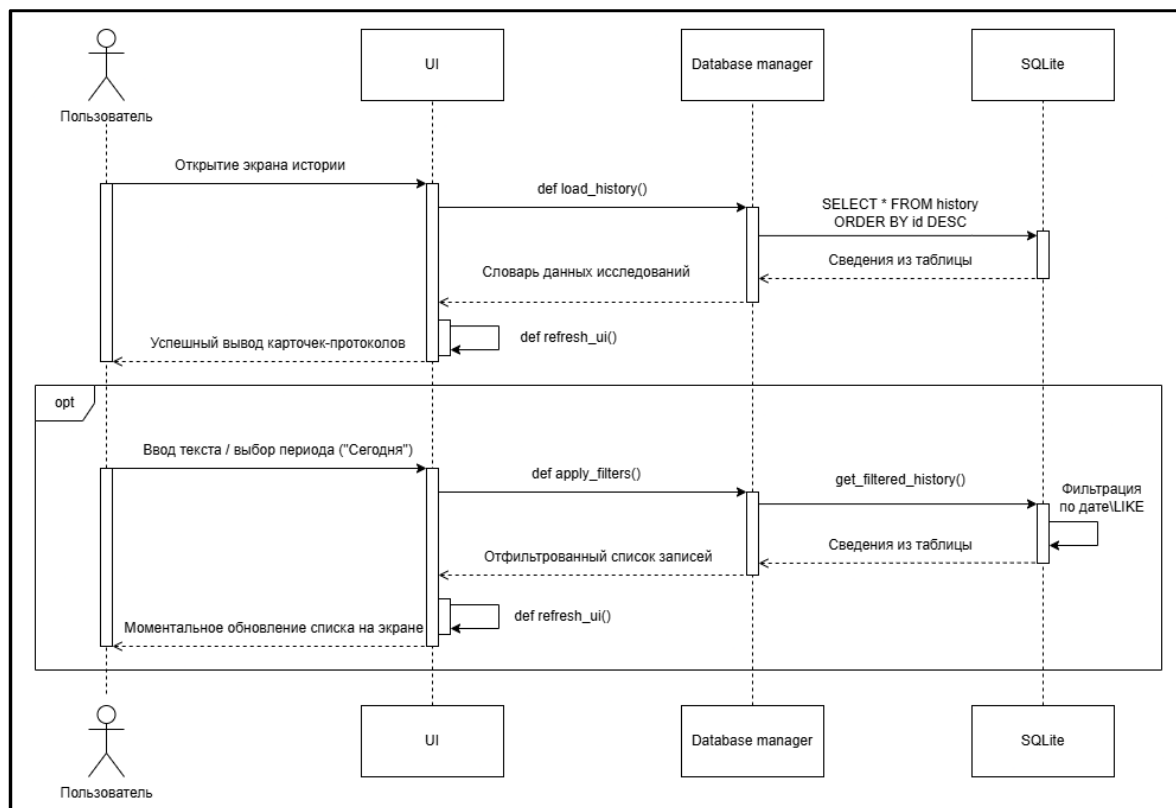


Рисунок 11 – Диаграмма последовательности прецедента посмотреть историю исследований

Краткое описание диаграммы последовательности представлено ниже.

Процесс отображения архива исследований начинается с того, что пользователь открывает экран истории исследований HistoryScreen в интерфейсе приложения. После этого интерфейсная часть автоматически формирует внутренний запрос к базе данных для извлечения всех ранее сохраненных записей. Получив запрос, локальная СУБД SQLite обращается к таблице history и извлекает накопленные метаданные. База данных выполняет чтение и возвращает интерфейсу массив записей, после чего интерфейс динамически генерирует на экране графические карточки-протоколы и уведомляет пользователя об успешном выводе архива.

Дополнительно в диаграмме отражен необязательный процесс (блок opt) оперативной фильтрации данных по временным периодам («Живой поиск»). В случае необходимости пользователь вводит поисковый запрос (номер протокола) или выбирает временной интервал («Сегодня», «За неделю»). Интерфейсная часть инициирует отдельный фильтрующий запрос к базе данных. Компонент `DatabaseManager` выполняет текстовый поиск по оператору `LIKE` средствами `SQL`, после чего осуществляет кадрирование временных периодов путем выгрузки данных в структуру `DataFrame` библиотеки `pandas` и возвращает в графический интерфейс оптимизированный массив словарей, получив статус успешного завершения поиска, мгновенно обновляет отображаемый список на экране пользователя.

Выводы по второй главе

Во второй главе было выполнено проектирование десктопного приложения для анализа изображений «Vision Pro AI». На основе анализа задачи были сформулированы функциональные и нефункциональные требования к программе.

Также была разработана диаграмма прецедентов, которая описывает роль эксперта и основные сценарии его работы с системой.

Для описания логики работы приложения были созданы диаграммы деятельности и последовательности в нотации `UML`. Они наглядно показывают процессы запуска нейросетей `YOLOv11` и `EasyOCR`, проверку результатов пользователем, а также поиск, фильтрацию и сохранение данных в локальную базу данных `SQLite`.

3. РЕАЛИЗАЦИЯ

3.1. Обоснование выбора инструментов разработки

Для разработки десктопного приложения «Vision Pro AI» был выбран язык программирования Python. Данный выбор обусловлен тем, что Python является стандартом в индустрии искусственного интеллекта и разработки интеллектуальных систем, обладая развитой экосистемой библиотек.

В качестве основных программных компонентов и фреймворков были выбраны:

- 1) `PyQt6` – кроссплатформенный фреймворк для создания графического интерфейса. Он предоставляет компонентную структуру, механизм сигналов и слотов для обмена данными и широкие возможности стилизации через `QSS`;
- 2) `YOLOv11` – современная сверточная нейросеть для высокоскоростной локализации объектов на изображениях в режиме реального времени;
- 3) `EasyOCR` – библиотека оптического распознавания символов, работающая автономно на локальном компьютере и стабильно поддерживающая кириллический сегмент;
- 4) `SQLite` – встраиваемая реляционная система управления базами данных, сохраняющая данные в один файл без необходимости развертывания сервера;
- 5) `ReportLab` – библиотека для программной верстки многостраничных отчетов в формате PDF.

3.2. Архитектурная структура и распределение модулей проекта

Архитектура разработанного десктопного приложения имеет выраженную модульную структуру и логически разделена на четыре основные функциональные подсистемы, каждая из которых решает свой обособленный круг задач. Взаимодействие между элементами этих подсистем строится на принципах слабой связанности, что позволяет изменять или дорабатывать отдельные модули (например, обновлять архитектуру нейросети) без необходимости переписывания интерфейсной части. Основными блоками системы выступают подсистема графического интерфейса, вычислительный ИИ-движок, модуль управления ресурсами, а также сервисы управления данными и формирования отчетности.

Для обеспечения высокой модульности, удобства отладки и строгого соблюдения выбранного трехслойного паттерна проектирования (Data-Engine-UI) все исходные файлы десктопного приложения «Vision Pro AI» сгруппированы в изолированные пакеты внутри главного каталога App. Логическое устройство файловой структуры разработанного программного комплекса имеет следующий вид:

- 1) App/ – корневой пакет, объединяющий все функциональные компоненты программы;
- 2) main.py – главная точка входа, отвечающая за инициализацию графической оболочки;
- 3) CFG/ – подсистема конфигурации, содержащая модуль config.py для динамического менеджмента путей к ресурсам;
- 4) Data/ – слой управления данными, представленный CRUD-менеджером database_manager.py;
- 5) Engine/ – вычислительное ядро системы, объединяющее модули искусственного интеллекта и аналитики (analytics_engine.py, image_processor.py, model_installer.py, object_detector.py, text_recognizer.py, vision_engine.py);

6) `ReportMaster/` – подсистема генерации отчетности, содержащая класс `report_generator.py`;

7) `Source/` – централизованное хранилище статических ассетов (шрифтов, иконок и таблиц стилей QSS);

8) `UI/` – слой графического интерфейса, разделенный на координатора `main_window.py`, пакет модальных окон `components/` и пакет экранов `screens/`.

Физическое распределение ответственности между разработанными программными модулями организовано по пяти основным направлениям:

1. Конфигурация и запуск системы. Файл `main.py` осуществляет первичный запуск приложения, настраивает параметры High-DPI для корректного масштабирования окон на экранах с высоким разрешением и вызывает менеджер интерфейса. Модуль `config.py` в автоматическом режиме вычисляет абсолютные пути ко всем каталогам на целевом компьютере. Это полностью исключает появление ошибок «файл не найден» при переносе программы между операционными системами и обеспечивает стабильный доступ к ресурсам из папки `Source`.

2. Компоненты искусственного интеллекта. Пакет `Engine` аккумулирует в себе всю вычислительную нагрузку приложения. Изолированные модули `object_detector.py` (инференс сверточной нейросети YOLOv11) и `text_recognizer.py` (движок EasyOCR) отвечают за непосредственное извлечение признаков из графических матриц раstra. Для обеспечения консистентности данных внутри аналитического конвейера, многоточечные полигоны координат EasyOCR программно приводятся к стандартному двухточечному формату прямоугольных ограничивающих рамок (`x1, y1, x2, y2`) путем вычисления экстремумов по осям абсцисс и ординат. Если на компьютере пользователя отсутствуют необходимые файлы весов нейросетей, модуль `model_installer.py` выполняет их безопасное скачивание в фоновом режиме. Центральный оркестратор `vision_engine.py` координирует вызовы этих моделей и агрегирует их ответы.

3. Вспомогательная обработка и аналитика. Внутри ИИ-ядра за графические трансформации отвечает компонент `image_processor.py`. Он производит нарезку фрагментов (кропов) подтвержденных объектов и осуществляет наложение финальных ограничивающих рамок. Модуль `analytics_engine.py` работает параллельно: он обрабатывает массивы данных, рассчитывает метрики эффективности и подготавливает информацию для построения графиков мониторинга.

4. Управление данными и хранением. Слой данных полностью инкапсулирован внутри модуля `database_manager.py`. Он скрывает в себе всю SQL-логику, открывает транзакции к локальному файлу базы данных SQLite и выполняет операции чтения, записи, поиска и удаления записей. Интерфейсная часть не имеет прямого доступа к таблицам базы данных, что защищает систему от случайного повреждения структуры данных.

5. Графическая оболочка и отчетность. Пакет UI управляет отображением оконных компонентов. Модуль `main_window.py` переключает рабочее пространства между экранами из папки `screens/` (главный экран, архив, аналитика) и вызывает всплывающие окна из папки `components/`. Завершающим звеном выступает модуль `report_generator.py`, который берет очищенные данные и выполняет автоматическую верстку печатных многостраничных документов в формате PDF.

Организация асинхронного взаимодействия модулей

При проектировании десктопных приложений, интегрированных с тяжелыми моделями глубокого обучения, ключевой инженерной проблемой является обеспечение отзывчивости графического интерфейса. Выполнение инференса сверточной нейросети YOLOv11 и запуск алгоритмов EasyOCR представляют собой ресурсоемкие вычислительные задачи, требующие значительного времени работы центрального или графического процессора. Выполнение этих операций в главном потоке приложения неизбежно приводит к его временной блокировке, зависанию оконных компонентов и аварийному завершению процесса операционной системой.

Для предотвращения данного эффекта взаимодействие между элементами графической оболочки и ядра искусственного интеллекта построено по принципу асинхронного делегирования задач с использованием механизма сигналов и слотов фреймворка `PyQt6`. Весь сквозной процесс обработки растровых матриц пикселей и передачи информации разделен на три последовательных этапа:

1. Инициализация и запуск фонового потока. При нажатии пользователем интерактивной кнопки запуска анализа слой интерфейса перехватывает событие клика. Чтобы исключить повторные запросы и защитить память от перегрузки, программа переводит кнопки управления главного окна в заблокированное состояние и выводит на экран анимированный индикатор загрузки. Одновременно с этим интерфейс инициализирует и запускает изолированный рабочий поток, наследуемый от класса `QThread`. Этот поток начинает выполняться в адресном пространстве оперативной памяти полностью независимо от главного GUI-потока, за счет чего окно приложения продолжает плавно работать, а системная анимация прогресс-бара не зависает.

2. Параллельные вычисления и потокобезопасная отправка данных. Выделенный фоновый поток перехватывает путь к графическому файлу и берет на себя всю математическую нагрузку по обращению к тензорам видеопамяти. Внутри потока последовательно или параллельно вызывается инференс сверточной модели `YOLOv11` для локализации геометрических координат объектов и извлекаются текстовые цепочки через `EasyOCR`. Связь между фоновым вычислителем и интерфейсом осуществляется бесконтактно. В процессе работы поток периодически генерирует промежуточные сигналы изменения прогресса, которые мгновенно перехватываются графическим слотом главного окна для обновления шкалы индикатора в режиме реального времени.

3. Агрегация результатов и финализация процесса. По завершении полного цикла распознавания фоновый модуль собирает извлеченные `bounding-boxes` и распознанные символы в единый структурированный

словарь, после чего испускает финальный сигнал успешного завершения. Главный поток приложения принимает этот сигнал, извлекает результаты анализа, полностью скрывает прогресс-бар и разблокирует элементы управления. На основе полученных данных система динамически перестраивает графические элементы экрана, выводя пользователю список найденных ИИ-меток в окне-редакторе. Сам фоновый поток после отправки данных безопасно уничтожается операционной системой, что полностью исключает утечки памяти.

Проектирование локальной базы данных и модуля аналитики

Для обеспечения надежного локального хранения информации и последующей оценки эффективности работы оператора в приложении спроектирована реляционная структура данных. Полноценное функционирование этого слоя обеспечивает встроенная система управления базами данных SQLite, которая сохраняет всю информацию в один файл и работает непосредственно внутри процесса приложения. Реализация данного функционала разделена на два ключевых компонента:

1. Проектирование схемы хранения данных. Вся история проведенных исследований аккумулируется в единой центральной таблице с именем `history`. Каждый кортеж данной таблицы содержит полный набор метаданных, описывающих один сеанс работы с изображением. В состав строки входит уникальный идентификатор записи в виде целочисленного первичного ключа с автоинкрементом для однозначной адресации строк, уникальный строковый номер экспертного заключения для быстрого сквозного поиска в архиве, а также текстовый штамп времени для фиксации точной даты проведения инференса.

Дополнительно в таблице сохраняется строка с первоначальным названием загруженного графического файла, адрес сохраненного на локальном диске кадра с наложенными финальными ИИ-рамками, текстовое поле большого объема для хранения агрегированного списка наименований

всех обнаруженных классов элементов, текстовое поле с извлеченными моделью EasyOCR символами и поле для текстовых комментариев оператора.

2. Реализация аналитического модуля. На основе накопленных в базе данных строк функционирует модуль аналитики, представленный компонентом `analytics_engine.py`. Его ключевой задачей является математический расчет ключевых показателей эффективности и подготовка массивов для визуализации. При запуске экрана статистики модуль считывает таблицу истории и передает массив записей в структуру библиотеки `pandas`. Использование табличных абстракций позволяет выполнять группировку, агрегацию и временное кадрирование за текущий день или неделю без создания сложных SQL-запросов.

Модуль автоматически вычисляет общее количество обработанных изображений, суммарное число найденных объектов и среднее время верификации кадра. Полученные структурированные ряды данных транслируются в графический движок `matplotlib`, который строит гистограммы распределения классов и линейные тренды интенсивности исследований, автоматически подстраивая цвета осей и фонов под палитру активной QSS-темы интерфейса.

Особенности реализации системы отчетности

Для фиксации результатов экспертных исследований и обеспечения возможности их отчуждения от программного комплекса в приложении реализована автономная подсистема выгрузки документов. Основным программным инструментом данного блока выступает модуль `report_generator.py`, функционирующий на базе кроссплатформенной библиотеки `ReportLab`. Процесс автоматического формирования выходных печатных форм построен без использования жестко размеченных графических бланков, что позволило реализовать гибкую потоковую структуру компоновки элементов. Архитектура построения и логика функционирования разработанной системы отчетности разделена на два ключевых аспекта:

1. Реализация конвейера потоковой верстки документов. Процесс генерации выходного протокола базируется на использовании специализированного шаблона `SimpleDocTemplate`, который организует последовательное заполнение пространства листа элементами из динамического массива. При инициализации процесса сборки отчета модуль принимает очищенные метаданные из локальной базы данных, включая уникальный номер протокола, временные штампы и текстовое содержимое.

Для интеграции графической информации система обращается к модулю `image_processor.py`, который подготавливает растровое изображение с нанесенными рамками нейросети YOLOv11 и конвертирует его в оптимизированный потоковый объект. Главным инженерным преимуществом такой архитектуры является динамический расчет высоты текстовых блоков. В случае если эксперт вносит объемные текстовые комментарии и заметки, система автоматически вычисляет дефицит свободного места на текущей странице и осуществляет корректный перенос избыточной текстовой информации на последующие листы документа, предотвращая наложение слоев и обрезание данных.

2. Обеспечение отказоустойчивости и локализации шрифтов. При проектировании генератора отчетов была решена проблема уязвимости стандартных XML-парсеров к управляющим символам. Поскольку текстовое содержимое, извлеченное моделью EasyOCR из реальных изображений, может содержать случайные знаки разметки или амперсанды, все строковые переменные перед отправкой на сборку проходят обязательную процедуру экранирования, что полностью исключает аварийное завершение процесса генерации.

Для обеспечения корректного отображения кириллического алфавита в подсистему внедрена процедура принудительной регистрации моноширинных векторных шрифтов NotoSans, статические файлы которых изолированы во внутреннем пакете ресурсов `Source`. Дополнительно на уровне генерации холста программно накладывается полупрозрачный водяной знак

с уникальным идентификатором исследования, выполняющий защитную функцию и подтверждающий подлинность сформированного экспертного заключения.

Выводы по третьей главе

В третьей главе была успешно выполнена программная реализация разработанной архитектуры десктопного приложения на языке программирования Python с использованием графического фреймворка PyQt6. В ходе работы была детально описана физическая структура пакетов и распределение обязанностей между модулями ИИ-движка, интерфейса и подсистемы хранения данных. Реализованный событийно-ориентированный механизм сигналов и слотов позволил полностью изолировать тяжелые вычисления нейросетей YOLOv11 и EasyOCR в отдельный асинхронный поток вычислений, за счет чего была решена проблема зависания графических окон интерфейса во время анализа изображений. Дополнительно в рамках главы была спроектирована схема локальной реляционной базы данных SQLite для персистентного хранения истории исследований и описан аналитический модуль на базе библиотеки pandas, рассчитывающий метрики эффективности оператора. Итоговым шагом стала реализация подсистемы генерации отчетов на базе ReportLab, которая обеспечивает автоматическую потоковую верстку многостраничных экспертных PDF-протоколов с полной поддержкой кириллицы и защитой от программных сбоев при парсинге спецсимволов.

4. ТЕСТИРОВАНИЕ

4.1. Автоматизированное тестирование

Автоматизированное тестирование вычислительной части десктопного приложения выполнено с использованием модульных и интеграционных тестов на базе фреймворка `pytest`. Тестовые сценарии позволяют имитировать программные вызовы оркестратора, контролировать логику конфигурации путей и проверять корректность работы методов обработки данных без необходимости ручного запуска графического интерфейса пользователя.

Для локализации тяжелых зависимостей и исключения сбоев динамической инициализации библиотек видеопамати на этапе сборки тестов применена технология `Mock`-объектов. Это позволило изолировать логику приложения и симитировать ответы ИИ-моделей в оперативной памяти компьютера.

Разработанный тестовый набор состоит из пяти функциональных блоков, которые покрывают следующие сценарии:

1. Тестирование подсистемы конфигурации на предмет динамического расчета абсолютных путей ко всем системным ресурсам, шрифтам и иконкам.
2. Проверка валидации полей структуры данных перед отправкой транзакций в базу данных и симуляция логики работы «живого поиска» по архиву исследований.
3. Верификация логики оркестрации и структуры текстовых данных результатов детекции и распознавания.
4. Проверка надежности работы `try-except` блока деградации вычислений при имитации отсутствия аппаратного ускорения видеокарты и автоматическом перенаправлении нагрузки на центральный процессор
5. Тестирование подсистемы отчетности на корректность экранирования спецсимволов для предотвращения аварийного завершения XML-парсера.

В ходе проведения автоматизированного тестирования все 24 разработанных тест-кейса были успешно пройдены за 0,09 секунды. Результаты прохождения автоматизированных тестов с выводом подробных отчетов по каждому шагу вычислений представлены на рисунке 12.

```
===== test session starts =====
platform win32 -- Python 3.12.5, pytest-9.0.3, pluggy-1.6.0 -- C:\...\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.12.5', 'Platform': 'Windows-10-10.0.19045-SP0',
'Packages': {'pytest': '9.0.3', 'pluggy': '1.6.0'},
'Plugins': {'anyio': '4.13.0', 'html': '4.2.0', 'metadata': '3.1.1', 'qt': '4.5.0'}}
PyQt6 6.7.1 -- Qt runtime 6.7.3 -- Qt compiled 6.7.1
rootdir: C:\CourseWork\Coursework
plugins: anyio-4.13.0, html-4.2.0, metadata-3.1.1, qt-4.5.0
collecting ... collected 24 items

tests/test_vision_system.py::test_config_resource_paths[...] PASSED [ 4%]
tests/test_vision_system.py::test_config_resource_paths[...] PASSED [ 8%]
tests/test_vision_system.py::test_config_resource_paths[...] PASSED [ 12%]
tests/test_vision_system.py::test_config_resource_paths[...] PASSED [ 16%]
tests/test_vision_system.py::test_database_save_operations[...] PASSED [ 20%]
tests/test_vision_system.py::test_database_save_operations[...] PASSED [ 25%]
tests/test_vision_system.py::test_database_save_operations[...] PASSED [ 29%]
tests/test_vision_system.py::test_database_search_logic[...] PASSED [ 33%]
tests/test_vision_system.py::test_database_search_logic[...] PASSED [ 37%]
tests/test_vision_system.py::test_database_search_logic[...] PASSED [ 41%]
tests/test_vision_system.py::test_vision_engine_orchestration_logic[...] PASSED [ 45%]
tests/test_vision_system.py::test_vision_engine_orchestration_logic[...] PASSED [ 50%]
tests/test_vision_system.py::test_vision_engine_orchestration_logic[...] PASSED [ 54%]
tests/test_vision_system.py::test_vision_engine_orchestration_logic[...] PASSED [ 58%]
tests/test_vision_system.py::test_vision_engine_orchestration_logic[...] PASSED [ 62%]
tests/test_vision_system.py::test_cuda_hardware_fallback_logic[True-cuda0] PASSED [ 66%]
tests/test_vision_system.py::test_cuda_hardware_fallback_logic[False-cpu0] PASSED [ 70%]
tests/test_vision_system.py::test_cuda_hardware_fallback_logic[True-cuda1] PASSED [ 75%]
tests/test_vision_system.py::test_cuda_hardware_fallback_logic[False-cpu1] PASSED [ 79%]
tests/test_vision_system.py::test_cuda_hardware_fallback_logic[False-cpu2] PASSED [ 83%]
tests/test_vision_system.py::test_report_generator_xml_escaping[...] [ 87%]
tests/test_vision_system.py::test_report_generator_xml_escaping[...] PASSED [ 91%]
tests/test_vision_system.py::test_report_generator_xml_escaping[...] PASSED [ 95%]
tests/test_vision_system.py::test_report_generator_xml_escaping[...] PASSED [100%]

===== 24 passed in 0.08s =====
```

Рисунок 12 – Логирование успешного выполнения сессии автоматизированного тестирования

4.2. Функциональное тестирование

Функциональное тестирование графической оболочки PyQt6 проведено ручным методом. В ходе испытаний успешно проверена сквозная цепочка действий пользователя: выбор файлов моделей, запуск сканирования с анимированным прогресс-баром, интерактивная разметка ИИ-меток в редакторе, сохранение логов в СУБД SQLite и выгрузка PDF-протокола через ReportLab. Поведение оконных компонентов полностью соответствует техническому заданию, а детальные сценарии тестирования вынесены в таблице 13 приложения Б.

ЗАКЛЮЧЕНИЕ

Целью данной курсовой работы являлась разработка десктопного приложения для автоматизированного интеллектуального анализа изображений с использованием искусственного интеллекта «Vision Pro AI». В ходе работы были выполнены следующие задачи:

- 1) выполнить анализ предметной области и обзор существующих аналогов;
- 2) сформулировать требования к десктопному приложению;
- 3) спроектировать десктопное приложения;
- 4) реализовать десктопное приложение;
- 5) провести тестирование разработанного десктопного приложения.

В результате создано локальное десктопное приложение для автоматизации процессов локализации объектов и распознавания символов. Программа обеспечивает гибкую настройку параметров ИИ-ядра, выбор весов нейросети YOLOv11 и ручную интерактивную верификацию найденных элементов перед архивацией.

Особое внимание уделено асинхронной архитектуре на базе фреймворка PyQt6, исключающей зависания интерфейса при нейросетевом инференсе. Реализованные модули расчета KPI на базе pandas и matplotlib, автоматического сбора датасетов и потоковой генерации PDF-протоколов в ReportLab существенно сокращают время обработки данных.

В дальнейшем проект может быть расширен за счет интеграции с облачными хранилищами, добавления модулей дообучения нейросетей YOLOv11 напрямую из интерфейса и внедрения алгоритмов обработки данных в режиме реального времени.

ЛИТЕРАТУРА

1. Афанасьева, Ж. С. Распознавание объектов с помощью сверточных нейронных сетей : учебное пособие / Ж. С. Афанасьева, А. Д. Афанасьев. – Иркутск : ИРНИТУ, 2023. – 134 с. – ISBN 978–5–8038–1914–1. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/497900> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.
2. Буч, Г. Язык UML. Руководство пользователя : руководство / Г. Буч, Д. Рамбо, И. Якобсон. – Москва : ДМК Пресс, 2008. – 496 с. – ISBN 5-94074-334-X. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/1246> (дата обращения: 04.04.2026). – Режим доступа: для авториз. пользователей.
3. Васильев М.Е., Шалимов А.С., Савина О.А. Обзор версий YOLO: одноэтапная модель сверточной нейронной сети. [Электронный ресурс]. URL: <https://7universum.com/ru/tech/archive/item/20293> (дата обращения: 01.04.2026).
4. Веб-сервис «Detectron». [Электронный ресурс]. URL: <https://detectron2.com>.
5. Веб-сервис «Roboflow». [Электронный ресурс]. URL: <https://roboflow.com>.
6. Галлатин К., Элбон К. Машинное обучение с использованием Python: Сборник рецептов. Практические решения от предобработки до глубокого обучения. 2023. 416 с.
7. Журнал Молодой ученый – Методы определения объектов на изображении / Я. Э. Живрин, Нафе Башар Алкзир. – Текст: непосредственный // Молодой ученый. – 2018. – № 7 (193). – С. 8–19. – URL: <https://moluch.ru/archive/193/48447> (дата обращения: 04.11.2025).
8. Запечников, С. В. Основы интеллектуального анализа данных и машинного обучения: Конспект лекций : учебное пособие / С. В. Запечников. – Москва : НИЯУ МИФИ, 2022. – 136 с. – ISBN 978–5–7262–2856–3. –

Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/355580> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

9. Крейман, Г. Биологическое и компьютерное зрение / Г. Крейман ; под редакцией Т. Б. Киселевой, Т. И. Люско ; перевод с английского И. Л. Люско. – Москва : ДМК Пресс, 2022. – 314 с. – ISBN 978–5–93700–100–9. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/241193> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

10. Малке, М. Конвейер ssos: Идентификация объектов Солнечной системы на астрономических изображениях / М. Малке, Э. Солано, Э. Буи, Б. Карри, Г. А. Вердос Клейн, Э. Бертен // *Astronomy and Computing*. – 2019. – Т. 28. – С. 100289. – DOI: 10.1016/j.ascom.2019.100289.

11. Мобильное приложение «Google Lens». [Электронный ресурс]. URL: <https://lens.google>.

12. Накамура, К. Оптимизация комбинаций обучающих изображений для обнаружения объектов на изображениях / К. Накамура, Р. Хама-саки, Ч. Ошима, К. Накаяма // *Lecture Notes in Computer Science*. – 2020. – Т. 12119. – С. 492-504. – DOI: 10.1007/978–3–030–50017–7_37.

13. Ненашев, В. А. Компьютерное зрение. Анализ, обработка и моделирование : учебное пособие / В. А. Ненашев. – Санкт–Петербург : ГУАП, 2022. – 78 с. – ISBN 978–5–8088–1806–4. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/341057> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

14. Обработка изображений с помощью OpenCV / Б. Г. Глория, Д. С. Оскар, Л. Э. Хосе, С. Г. Исмаэль. – Москва : ДМК Пресс, 2016. – 210 с. – ISBN 978–5–97060–387–1. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/90116> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

15. Орлов, А. А. Применение преобразования Хоха для обнаружения и идентификации объектов на изображениях / А. А. Орлов, Е. М. Залазинская // Методы и устройства передачи и обработки информации. – 2002. – № 2. – С. 103-107. – EDN NRMOJT. – Текст : электронный // Научная электронная библиотека eLIBRARY. – URL: <https://elibrary.ru/item.asp?id=16244711> (дата обращения: 04.12.2025). – Режим доступа: для авториз. пользователей.

16. Хоменко Т.В., Иргалиев А.А., Тараканов В.Д. Моделирование процесса распознавания символов в нормативных документах организации. [Электронный ресурс]. URL: <https://vestnik.astu.org/ru/nauka/article/64121/view> (дата обращения: 01.04.2026).

17. Шапиро, Л. Компьютерное зрение: учебное пособие / Л. Шапиро, Д. Стокман ; перевод с английского А. А. Богуславского под редакцией С. М. Соколова. – 5–е изд. (эл.). – Москва: Лаборатория знаний, 2024. – 763 с. – ISBN 978–5–93208–725–1. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/417998> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

18. Ян, Э. С. Программирование компьютерного зрения на языке Python / Э. С. Ян ; перевод с английского А. А. Слинкин. – Москва : ДМК Пресс, 2016. – 312 с. – ISBN 978–5–97060–200–3. – Текст : электронный // Лань : электронно–библиотечная система. – URL: <https://e.lanbook.com/book/93569> (дата обращения: 04.11.2025). – Режим доступа: для авториз. пользователей.

19. Fernández A.V. Mastering OpenCV 4 with Python. Packt Publishing, 2024. 532 с.

ПРИЛОЖЕНИЯ

Приложение А. Спецификация вариантов использования

Таблица 1 – Спецификация ВИ «Выбрать исходное фото»

Прецедент: Выбрать исходное фото
ID: 1
Краткое описание: Пользователь выбирает изображение с локального диска
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Приложение «Vision Pro AI» запущено, на экране отображается окно идентификатора.
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает на графический элемент выбора изображения. 2. Система вызывает стандартное диалоговое окно операционной системы для обзора локальных файлов. 3. Пользователь осуществляет навигацию по директориям диска и выбирает необходимый графический файл в формате JPEG, PNG. 4. Пользователь нажимает кнопку «Открыть» в системном диалоговом окне. 5. Система проверяет расширение и валидность выбранного файла. 6. Система автоматически копирует выбранный растр во внутренний изолированный каталог архива (Results/ResultsImages/), присваивая файлу уникальное имя на основе текущего временного штампа. 7. Система отображает загруженное изображение в центральной графической зоне интерфейса приложения.
Постусловия: 1. Выбранный графический файл успешно кэширован в файловой системе приложения. 2. Путь к кэшированному изображению сохранен во временном контексте текущей рабочей сессии для передачи в ИИ-движок.
Альтернативные потоки: A1. Выбран файл неподдерживаемого формата или поврежден: 1. Система блокирует дальнейшую обработку. 2. Система выводит на экран предупреждающее сообщение «Ошибка: неверный формат файла». 3. Возврат к шагу 1 основного потока.

Таблица 2 – Спецификация ВИ «Настроить параметры ИИ»

Прецедент: Настроить параметры ИИ
ID: 2
Краткое описание: Пользователь изменяет конфигурацию и чувствительность нейросетевых моделей перед запуском инференса.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Окно идентификатора приложения активно, модули нейросетей успешно инициализированы в кодовой базе.
Основной поток: 1. Прецедент начинается, когда Пользователь раскрывает выпадающий список доступных ИИ-моделей. 2. Система динамически сканирует внутреннюю директорию Source/Models/ и выводит список всех обнаруженных файлов весов формата .pt. 3. Пользователь выбирает необходимую конфигурацию модели (например, YOLOv11n или YOLOv11s) из списка. 4. Пользователь перемещает ползунок (слайдер) регулировки порога точности (Confidence Threshold), устанавливая желаемое процентное ограничение. 5. Пользователь при необходимости активирует чекбокс режима «Только текст» для отключения детектора объектов. 6. Система мгновенно фиксирует измененные значения в конфигурационном объекте сессии.
Постусловия: 1. Новые параметры чувствительности и выбранная модель зафиксированы в памяти приложения и готовы к передаче в фоновый рабочий поток анализа.
Альтернативные потоки: Нет

Таблица 3 – Спецификация ВИ «Запустить анализ»

Прецедент: посмотреть Запустить анализ
ID: 3
Краткое описание: Пользователь запускает процесс сквозного нейросетевого распознавания графических образов и печатных символов на выбранном изображении.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Исходное изображение успешно выбрано (выполнено ВИ-01), параметры ИИ сконфигурированы (выполнено ВИ-02).
Основной поток: 1. Прецедент начинается, когда Пользователь загружает изображение.

Прецедент: Запустить анализ
Основной поток: 2. Система блокирует элементы управления окна идентификатора и запускает асинхронный фоновый поток вычислений <code>AnalysisWorker</code> (<code>QThread</code>). 3. Система выводит на экран анимированный индикатор загрузки (прогресс-бар). 4. В фоновом потоке Система вызывает инференс нейросети YOLOv11 для детекции объектов и извлечения координат рамок (<code>bounding boxes</code>). 5. В фоновом потоке Система одновременно вызывает инференс EasyOCR для распознавания печатных символов (если не включен режим «Только текст»). 6. Система производит математическое слияние и унификацию всех извлеченных координат в единый плоский массив формата <code>[x1, y1, x2, y2]</code>. 7. По завершении вычислений фоновый поток передает структурированный словарь результатов в главный UI-поток и завершает работу. 8. Система скрывает индикатор загрузки и отображает изображение с найденными объектами.
Постусловия: 1. Результаты интеллектуального распознавания успешно извлечены и выведены в интерфейс для последующего расширенного сценария «Сформировать протокол».
Альтернативные потоки: A1. Отсутствует аппаратное ускорение CUDA: 1. Система перехватом исключения <code>try-except</code> фиксирует отсутствие совместимой видеокарты. 2. Система автоматически перенаправляет вычислительную нагрузку инференса на центральный процессор (CPU). 3. Основной поток продолжается с шага 4. A2. В процессе анализа обнаружен поврежденный файл модели: 1. Фоновый поток аварийно завершается. 2. Система удаляет поврежденный файл весов, выводит сообщение об ошибке и осуществляет автоматический откат на базовую стабильную модель из кэша.

Таблица 4 – Спецификация ВИ «Открыть менеджер моделей»

Прецедент: Открыть менеджер моделей
ID: 4
Краткое описание: Пользователь открывает специализированный программный модуль для управления локальными файлами весов нейросетей YOLO.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Приложение запущено, окно идентификатора программы активно.
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает кнопку «Менеджер моделей» в панели управления. 2. Система инициализирует и выводит на экран диалоговое окно <code>YoloInstallerDialog</code>

Прецедент: Открыть менеджер моделей
Основной поток: 3. Система считывает содержимое директории <code>Source/Models/</code> и выводит текущий список установленных и доступных для загрузки моделей. 4. Пользователь изучает состояние локального хранилища весов нейросетей. 5. Пользователь закрывает диалоговое окно, завершая прецедент.
Постусловия: 1. Локальное состояние доступных ИИ-моделей успешно визуализировано для Пользователя.
Альтернативные потоки: Нет.

Таблица 5 – Спецификация ВИ «Посмотреть историю исследований»

Прецедент: Посмотреть историю исследований
ID: 5
Краткое описание: Пользователь открывает архив выполненных ранее анализов для просмотра накопленных данных, фильтрации и выгрузки отчетов.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Приложение запущено, локальная база данных SQLite успешно подключена и доступна для чтения.
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает на вкладку «История исследований» в главном меню. 2. Пользователь нажимает кнопку «Обновить данные» 3. Система отправляет SQL-запрос к локальной базе данных SQLite для извлечения всех сохраненных протоколов. 4. Система трансформирует полученные кортежи данных в объекты словарей с помощью фабрики строк (<code>sqlite3.Row</code>). 5. Система динамически генерирует в интерфейсе список закругленных графических карточек-протоколов, содержащих дату, имя файла и краткие результаты детекции. 6. Пользователь осуществляет просмотр архива и использует строку поиска для мгновенной «живой» фильтрации записей. 7. Пользователь переключается на другой экран, закрывая архив
Постусловия: 1. Содержимое локальной базы данных успешно выведено на экран в структурированном графическом виде.
Альтернативные потоки: Нет.

Таблица 6 – Спецификация ВИ «Сформировать протокол»

Прецедент: Сформировать протокол
ID: 6
Краткое описание: Пользователь принимает осознанное решение зафиксировать и сохранить результаты текущей сессии ИИ-анализа в локальную базу данных.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Успешно завершён автоматический инференс нейросетей (выполнено ВИ-03).
Основной поток: 1. Прецедент начинается автоматически в качестве расширения сценария анализа, когда Пользователь вносит текстовый комментарий или экспертную заметку в выделенное текстовое поле диалогового окна 2. Пользователь нажимает кнопку «Редактировать и сохранить» 3. Система открывает окно интерактивного редактора <code>ObjectEditorDialog</code> , перед Пользователем открывается список найденных ИИ объектов и текстовых блоков. 4. Пользователь вручную отмечает чекбоксами те элементы, которые он считает корректными, и снимает отметки с ложных срабатываний нейросети. 5. Пользователь при необходимости нажимает кнопки группового управления («Выделить все» / «Снять все») 6. Пользователь нажимает кнопку «Применить и сохранить протокол». 7. Система открывает окно ввода номера протокола. 8. Пользователь вводит номер протокола и нажимает кнопку «ОК» 7. Система отправляет проверенные координаты объектов в модуль <code>ImageProcessor</code> , который выполняет нарезку фрагментов (кропов) и сохраняет их в локальный датасет. 8. Система формирует структурированную запись (дата, имя файла, массив утвержденных объектов, текст, заметки) и выполняет <code>INSERT</code> -запрос в локальную базу данных <code>SQLite</code> . 9. Система закрывает окно редактора и производит повторную отрисовку финального кадра с нанесением контуров только одобренных пользователем объектов. 10. Система выводит сообщение об успешном сохранении. 11. Система меняет текст кнопки «Редактировать и сохранить» на «Сохранено в бд».
Постусловия: 1. Утвержденные результаты анализа надежно записаны в СУБД <code>SQLite</code> . 2. Локальный датасет пополнен новыми графическими кропами объектов.
Альтернативные потоки: A1. Отмена сохранения протокола: 1. На шаге 5 основного потока Пользователь нажимает кнопку «Отмена» или закрывает окно редактора на крестик. 2. Система полностью игнорирует результаты текущего инференса и не вносит никаких записей в <code>SQLite</code> . 3. Временные кэш-файлы растра зачищаются, окно закрывается, прецедент завершается.

Таблица 7 – Спецификация ВИ «Удалить протокол исследования»

Прецедент: Удалить протокол исследования
ID: 7
Краткое описание: Пользователь безвозвратно удаляет выбранную запись об исследовании из архива с одновременным уничтожением связанных файлов на диске.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Пользователь находится на экране «История исследований» (выполнено ВИ-05), в списке отображается как минимум одна карточка протокола.
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает на карточку конкретного протокола. 2. Система выводит на экран протокол исследования. 3. Пользователь нажимает кнопку «Удалить протокол». 4. Система выводит на экран модальное диалоговое окно с запросом на подтверждение удаления: «Вы уверены, что хотите удалить этот протокол?». 5. Пользователь нажимает кнопку «YES». 6. Система формирует и отправляет SQL-команду <code>DELETE FROM history WHERE id = ?</code> в локальную СУБД SQLite. 7. База данных стирает строку с метаданными протокола и возвращает подтверждение успешной транзакции. 8. Система считывает путь к сохраненному изображению из контекста и обращается к локальной файловой системе 9. Система обновляет графический интерфейс экрана истории, каскадно удаляя карточку с экрана.
Постусловия: 1. Запись полностью стерта из базы данных SQLite..
Альтернативные потоки: A1. Отмена удаления пользователем: 1. На шаге 5 основного потока Пользователь нажимает кнопку «NO» или закрывает диалоговое окно. 2. Система закрывает модальное окно без выполнения SQL-запросов. 3. Данные в БД и файлы на диске остаются без изменений. Прецедент завершается

Таблица 8 – Спецификация прецедента «Сгенерировать PDF-отчет»

Прецедент: Сгенерировать PDF-отчет
ID: 8
Краткое описание: Пользователь запускает экспорт данных выбранного протокола в официальный многостраничный документ формата PDF для печати или отправки.
Главные актеры: Пользователь
Второстепенные актеры: Нет

Прецедент: Сгенерировать PDF-отчет
Предусловия: 1. Пользователь открыл экран истории (выполнено ВИ-05).
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает на карточку конкретного протокола. 2. Система выводит на экран протокол исследования. 3. Пользователь нажимает кнопку «Экспорт в PDF». 4. Пользователь с помощью операционного проводника выбирает место сохранения PDF протокола. 5. Система извлекает полный набор данных протокола (дата, имя файла, список объектов, OCR-текст, экспертные заметки) в виде словаря. 6. Система вызывает программный модуль верстки на базе библиотеки ReportLab. 7. Система автоматически экранирует управляющие XML-символы в тексте заметок пользователя для предотвращения сбоя парсера. 8. Система рассчитывает высоту текстовых блоков и формирует многостраничный макет, автоматически перенося длинные записи на новые листы. 9. Система накладывает на задний план каждой страницы полупрозрачное изображение фирменного логотипа (водяной знак). 10. Система компилирует итоговый файл и сохраняет его в указанную пользователем директорию. 11. Система выводит на экран уведомление об успешной генерации и автоматически открывает документ в системной программе просмотра PDF.
Постусловия: 1. В указанной директории пользователем сохранен файл отчета, содержащий всю информацию об исследовании.
Альтернативные потоки: A1. Ошибка доступа к файловой системе: 1. Если целевой файл PDF открыт в другой программе и заблокирован, Система ловит исключение <code>PermissionError</code>. 2. Система выводит сообщение: «Ошибка сохранения: закройте файл отчета, если он открыт в другой программе». 3. Прецедент аварийно завершается без перезаписи файла.

Таблица 9 – Спецификация ВИ «Экспортировать статистику в Excel»

Прецедент: Экспортировать статистику в Excel
ID: 9
Краткое описание: Пользователь осуществляет выгрузку всей накопленной истории исследований в единую сводную таблицу формата Excel для ведения аналитики.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: Локальная база данных SQLite содержит выполненные записи протоколов, открыт экран аналитики.

Прецедент: Экспортировать статистику в Excel
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает кнопку «Сводный Excel отчет» в нижней панели управления. 2. Пользователь с помощью операционного проводника выбирает место сохранения Excel отчета. 3. Система отправляет глобальный SQL-запрос на чтение всей таблицы истории из SQLite. 4. Система формирует структуру таблицы формата .xlsx при помощи встроенного табличного движка. 5. Система заполняет строки таблицы данными: номер протокола, дата, имя исходного файла, количество объектов, распознанный текст. 6. Система компилирует итоговый файл и сохраняет его в указанную пользователем директорию. 8. Система выводит всплывающее окно «Экспорт успешно завершен».
Постусловия: 1. На локальном диске компьютера создан сводный отчет в таблице Excel.
Альтернативные потоки: A1. База данных пуста: 1. На шаге 2 Система обнаруживает, что в таблице истории нет записей. 2. Система блокирует создание файла Excel и выводит предупреждение: «Архив пуст, экспорт невозможен». 3. Прецедент завершается. A2. Ошибка доступа к файловой системе: 1. Если целевой файл Excel открыт в другой программе и заблокирован, Система ловит исключение <code>PermissionError</code>. 2. Система выводит сообщение: «Ошибка сохранения: закройте файл отчета, если он открыт в другой программе». 3. Прецедент аварийно завершается без перезаписи файла.

Таблица 10 – Спецификация ВИ «Скачать веса YOLO»

Прецедент: Скачать веса YOLO
ID: 10
Краткое описание: Пользователь запускает фоновое скачивание предобученных весов нейросети YOLO нужного масштаба из открытого удаленного репозитория Hugging Face.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: Открыто диалоговое окно «Менеджер моделей» (выполнено ВИ-04), компьютер пользователя подключен к сети Интернет, провайдер не блокирует доступ
Основной поток: 1. Прецедент начинается, когда Пользователь выбирает из списка доступных сетевых моделей нужную конфигурацию и нажимает кнопку «Скачать»..

Прецедент: Скачать веса YOLO
2. Система инициализирует асинхронный фоновый поток <code>ModelInstaller</code> (<code>QThread</code>), передавая ему URL-адрес файла весов 3. Система блокирует кнопку скачивания в интерфейсе менеджера для предотвращения повторных запросов. 4. Фоновый поток отправляет безопасный HTTPS-запрос к серверам репозитория Hugging Face. 5. По мере получения пакетов данных фоновый поток рассчитывает прогресс скачивания и отправляет Qt-сигналы в интерфейс. 6. Система принимает сигналы потока и динамически обновляет графический прогресс-бар в строке выбранной модели. 7. По завершении скачивания Система сохраняет бинарный файл весов формата <code>.pt</code> во внутренний каталог <code>Source/Models/</code>. 8. Система разблокирует интерфейс и меняет статус модели на «Установлено». Выпадающий список на главном экране автоматически обновляется.
Постусловия: 1. Новый файл весов нейросети успешно загружен, проверен и готов к использованию в инференсе.
Альтернативные потоки: A1. Отсутствует интернет-соединение или сервер недоступен: 1. На шаге 4 фоновый поток фиксирует сетевую ошибку (<code>ConnectionError</code>). 2. Фоновый поток аварийно завершается и отправляет сигнал об ошибке в UI-поток. 3. Система скрывает прогресс-бар, разблокирует интерфейс и выводит сообщение: «Ошибка сети: проверьте подключение к Интернету». 4. Поврежденный недокачанный файл автоматически удаляется с диска.

Таблица 11 – Спецификация ВИ «Загрузить кастомную модель»

Прецедент: Загрузить кастомную модель
ID: 11
Краткое описание: Пользователь импортирует сторонний, самостоятельно обученный файл весов нейросети формата <code>.pt</code> по ссылке <code>GitHub/raw/.pt</code>
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: Открыто диалоговое окно «Менеджер моделей» (выполнено ВИ-04), компьютер пользователя подключен к сети Интернет, провайдер не блокирует доступ
Основной поток: 1. Прецедент начинается, когда Пользователь нажимает кнопку «Импортировать кастомную модель». 2. Система инициализирует асинхронный фоновый поток <code>ModelInstaller</code> (<code>QThread</code>), передавая ему URL-адрес файла весов 3. Система блокирует кнопку скачивания в интерфейсе менеджера для предотвращения повторных запросов. 4. Фоновый поток отправляет безопасный HTTPS-запрос к серверам репозитория GitHub.

Прецедент: Загрузить кастомную модель
Основной поток: 5. По мере получения пакетов данных фоновый поток рассчитывает прогресс скачивания и отправляет Qt-сигналы в интерфейс. 6. Система принимает сигналы потока и динамически обновляет графический прогресс-бар в строке выбранной модели. 7. По завершении скачивания Система сохраняет бинарный файл весов формата .pt во внутренний каталог Source/Models/. 8. Система разблокирует интерфейс и меняет статус модели на «Установлено». Выпадающий список на главном экране автоматически обновляется.
Постусловия: 1. Кастомная модель успешно интегрирована в экосистему приложения и доступна для полноценного инференса.
Альтернативные потоки: A1. Файл имеет неверную структуру или расширение: 1. На шаге 4 Система обнаруживает, что файл не является валидной моделью PyTorch. 2. Система блокирует импорт и выводит ошибку: «Неверный формат кастомной модели.». 3. Прецедент завершается без скачивания файла.

Таблица 12 – Спецификация ВИ «Удалить файл модели»

Прецедент: Удалить файл модели
ID: 12
Краткое описание: Пользователь удаляет файлы модели.
Главные актеры: Пользователь
Второстепенные актеры: Нет
Предусловия: 1. Открыто диалоговое окно «Менеджер моделей» (выполнено ВИ-04), в списке установленных присутствует как минимум одна модель (кроме базовой встроенной).
Основной поток: 1. Прецедент начинается, когда Пользователь выбирает одну из ранее установленных моделей в списке менеджера. 2. Пользователь нажимает на кнопку «Удалить» напротив имени файла. 3. Система проверяет, не является ли данная модель активной в текущей сессии главного окна. 4. Система выполняет физическое удаление бинарного файла .pt из папки Source/Models/ средствами встроенных библиотек операционной системы. 5. Система мгновенно обновляет интерфейс менеджера моделей, убирая удаленный элемент из списка. 6. Система каскадно обновляет выпадающий список выбора ИИ-моделей на главном экране.
Постусловия: 1. Файл модели удален.
Альтернативные потоки: Нет.

Приложение Б. Результаты функционального тестирования

Таблица 13 – Функциональное тестирование

№	Название теста	Начальные условия	Шаги тестирования	Ожидаемый результат	Статус
1	Выбор конфигурации ИИ	Приложение запущено, открыт экран идентификатора.	1. Выбрать доступную модель. 2. Изменить статус на нахождения текста. 3. Изменить порог уверенности	Система считывает файлы с диска, фиксирует параметры в памяти и обновляет настройки.	Пройден
2	Асинхронный инференс	Выбрано изображение для анализа.	1. Нажать кнопку анализа. 2. Попробовать перейти на другой экран.	Кнопки отмены и анализа блокируются, запускается шкала прогресса. Окно плавно перемещается без зависаний. Доступен переход на другие экраны..	Пройден
3	Экспертная верификация	Анализ завершен	1. Ввод текста в поле заметок. 2. Нажать сохранить протокол. 3. Выбрать нужные объекты. 4. Сохранить.	Невыбранный объект удаляется из структуры, пользовательский комментарий привязывается к логу.	Пройден
4	Экспорт экспертных PDF	Открыт экран выбранного протокола.	1. Нажать кнопку «Экспорт Excel». 2. Проверить созданный файл на диске.	Автоматическая потоковая верстка отчета, длинный текст заметок корректно переносится на лист 2.	Пройден
5	Экспорт сводки данных Excel	Открыт аналитический экран со статистикой.	1. Нажать кнопку «Экспорт сводки в Excel». 2. Проверить созданный файл на диске.	Модуль аналитики собирает агрегированные KPI из pandas и успешно выгружает отчет в формате .xlsx.	Пройден